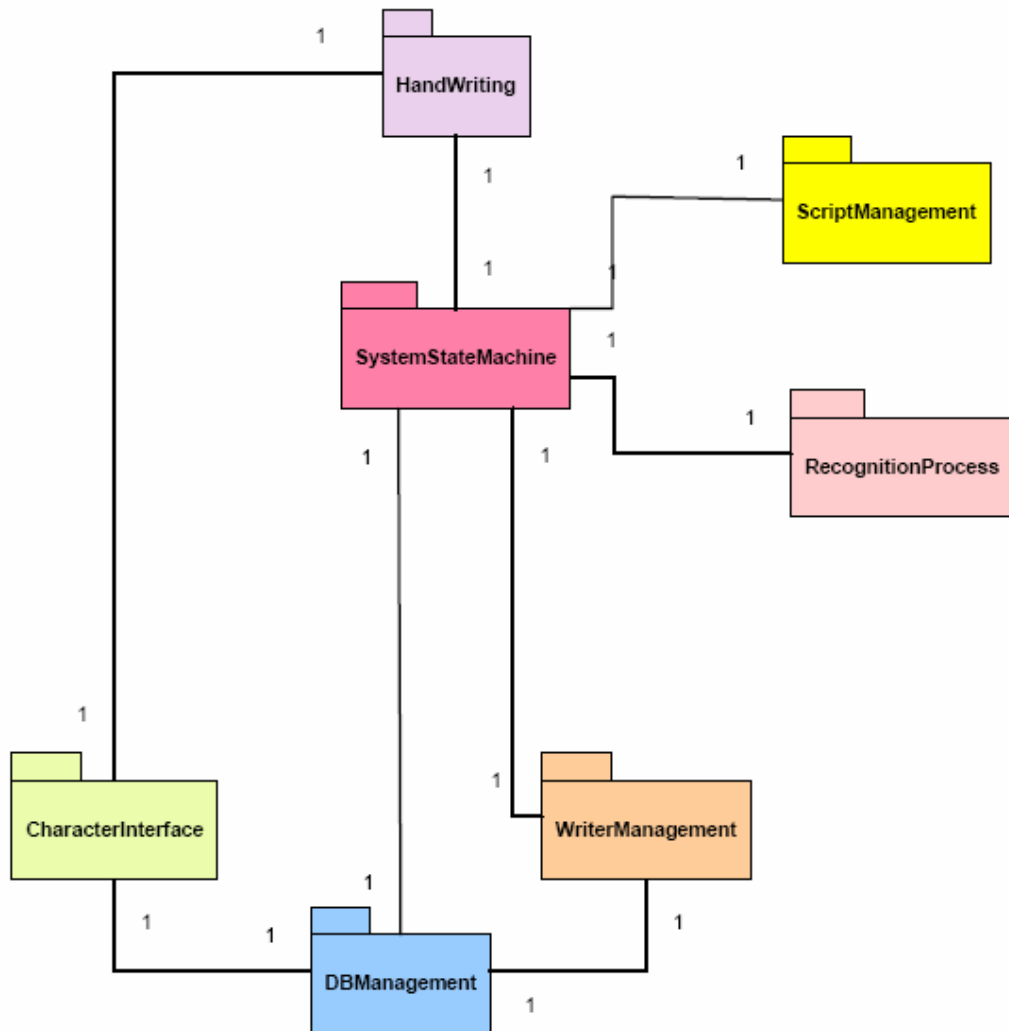


บทที่ 4

การออกแบบและพัฒนา

4.1 สถาปัตยกรรมซอฟต์แวร์ของระบบอ่านลายมือเขียนภาษาไทย



รูปที่ 4.1 แสดงความสัมพันธ์ระหว่างnamespace ต่างๆในระบบ

4.1.1 ระบบอ่านลายมือเขียนภาษาไทย ประกอบด้วยโมดูลการทำงาน 4 ส่วนหลักดังนี้

1. ส่วนติดต่อกับผู้ใช้งานระบบ (user interface)
2. ส่วนโปรแกรมหลัก (business process)
3. ส่วนติดต่อกับฐานข้อมูล (storage)
4. ส่วนจัดเก็บข้อมูลลงไฟล์เอกสาร (storage)

4.1.2 Namespace ในระบบ

- Handwriting**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่จัดการติดต่อกับผู้ใช้ระบบ ผ่านการรับข้อมูลจากผู้ใช้ระบบ และแสดงผลข้อมูลให้ผู้ใช้ระบบ ออกทางหน้าจอ
- ScriptManagement**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่จัดการกับอักษรต่างๆที่ต้องการให้ผู้ใช้ระบบเขียนตาม (อักษรสคริปต์)
- RecognitionProcess**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ โดยจัดการติดต่อกับระบบของโครงงานเดิมเพื่อแปลงตัวอักษรลายมือเขียนเป็นอักษรโคด
- SystemStateMachine**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ซึ่งจัดการตัวอักษรที่ไม่เป็นอักษรโคด และเป็นส่วนหลักของระบบในการควบคุมการประมวลผลโปรแกรม คล้าย main function
- WriterManagement**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่จัดการข้อมูลของผู้ใช้ระบบ โดยหากผู้ใช้ระบบเป็นผู้ใช้ใหม่ระบบจะเก็บข้อมูลของผู้ใช้นั้นเข้าฐานข้อมูล
- CharacterInterface**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่เป็นสื่อกลางในการแปลงรหัสอักษรระหว่างอักษรจากฐานข้อมูลและอักษรที่ใช้แสดงผลทางหน้าจอ (แปลงอักษรระหว่างรหัส ASCII และ รหัส UNICODE)
- DBManagement**: รวบรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ติดต่อกับฐานข้อมูล

4.1.3 Source Code ของระบบ ประกอบด้วยไฟล์ต่างๆดังนี้

Header File	Source File	Namespace	Class
Form1.h	Form1.cpp	Handwriting	1. Form1
MyControl.h	MyControl.cpp	Handwriting	1. MyControl
MyControlLine.h	MyControlLine.cpp	Handwriting	1. LineAbstraction, 2. DistanceBetween- WrittenLine
MyUI.h	MyUI.cpp	Handwriting	1. MyUI
PrototypeControl.h	PrototypeControl.cpp	Handwriting	1. PrototypeControl
PrototypeDisplay.h	PrototypeDisplay.cpp	Handwriting	1. PrototypeDisplay
CharacterDialogBox.h	CharacterDialogBox.cpp	Handwriting	1. CharacterDialogBox
DbUI.h	DbUI.cpp	Handwriting	1. DbUI
DbView.h	DbView.cpp	Handwriting	1. DbView

Header File	Source File	Namespace	Class
NewWriterDialog.h	NewWriterDialog.cpp	Handwriting	1. NewWriterDialog
NewWriterUI.h	NewWriterUI.cpp	Handwriting	1. NewWriterUI
WriterUI.h	WriterUI.cpp	Handwriting	1. WriterUI
DisplayWriterIdDialog.h	DisplayWriterIdDialog.cpp	Handwriting	1. DisplayWriterId-Dialog
SystemStateMachine.h	SystemStateMachine.cpp	SystemStateMachine	1. ProcessingMachine
SystemStateContent.h	SystemStateContent.cpp	SystemStateMachine	1. RemovedStated-Message, 2. RecognItem, 3. RecognList, 4. StateContent, 5. StateStack, 6. SystemState-StackInterface, 7. SystemStateStack,
SystemStateTable.h	SystemStateTable.cpp	SystemStateMachine	២៧
ScriptProcess.h	ScriptProcess.cpp	ScriptManagement	1. ScriptProcess
Recognition.h	Recognition.cpp	RecognitionProcess	1. PrototypeCharacter, 2. PrototypeList, 3. Recognition-Interface, 4. RecognitionProcess
CharacterInterface.h	CharacterInterface.cpp	CharacterInterface	1. CharacterInterface
ConstantIdentifier.h	ConstantIdentifier.cpp	ConstantIdentifier	២៧
DBConnection.h	DBConnection.cpp	DBManagement	1. DBConnection
DBContent.h	DBContent.cpp	DBManagement	1. Writer, 2. CharacterDB
StringTransformation.h	StringTransformation.cpp	StringTransformation	២៧
WriterManagement.h	WriterManagement.cpp	WriterManagement	1. WriterInterface, 2. WriterProfile

Header File	Source File	Namespace	Class
CurveList.h	CurveList.cpp	ไม่มี Namespace	1. CurveInfo, 2. CurveList
CurveListBuilder.h	CurveListBuilder.cpp	ไม่มี Namespace	1. CurveListBuilder
MousePoint.h	MousePoint.cpp	ไม่มี Namespace	1. Point, 2. PointList, 3. Vector

4.2 Class และ Interface ของคลาส

4.2.1. Handwriting: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดการเกี่ยวกับการติดต่อกับผู้ใช้งานระบบ การรับข้อมูลจากผู้ใช้งานระบบ และแสดงผลข้อมูลออกทางหน้าจอ

4.2.1. 1. Class Form เป็นคลาสจัดการกับ user interface สร้าง instance ของคลาสดังๆ และสร้างความสัมพันธ์เชื่อมโยงระหว่าง instance เหล่านั้น เมื่อปิดโปรแกรม instance ต่างๆ ที่ถูกสร้างขึ้นจะถูกกำจัดโดย default destructor ของคลาสนี้

```

public ref class Form1 : public System::Windows::Forms::Form
{
private:
    MyUI^ myUI ;
    MyControl^ myControl;

    PrototypeControl^ protoControl;

    ProcessManagement* processMan;
    SystemStateMachine::SystemStateStackInterface*
stackMan;
    RecognitionProcess::RecognitionInterface* recogMan;
    WriterManagement::WriterInterface* writerIn;
    LineAbstraction* disLine;
    SystemStateMachine::ProcessingMachine* coreMan;
    ScriptManagement::ScriptProcess* scriptMan;
    CurveListBuilder* curveLst;
private:
    void characterDBClick(System::Object^ sender,
System::EventArgs^ e);
    void writerDBClick(System::Object^ sender,
System::EventArgs^ e);
    void newWriterClick(System::Object^ sender,
System::EventArgs^ e);

```

4.2.1.2. Class WritingPadManagement (class LineAbstraction) เป็น Abstract

Class จัดการกับเส้นบรรทัดใน Writing Pad โดย class ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class LineAbstraction
{
public:
    virtual void increaseLineIndex() = 0;
    virtual std::pair<int, int> getLinePair()= 0;
    virtual std::pair<int, int> getLinePair(unsigned index)= 0;
    virtual void setMyControl(HandWriting::MyControl^ mycon) = 0;
    virtual void createNewLine() = 0;
    virtual unsigned getLineIndex() const = 0;
    virtual unsigned getLineCount() = 0;
    virtual int getSpaceLine() const = 0;
    virtual void setSpaceLine(unsigned mul)= 0;
    virtual void clearWritingPad() = 0;
};
```

และ คลาสที่สืบทอด (Inherit) มาจากAbstract Class ข้างต้นประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class DistanceBetweenWrittenLine :public LineAbstraction
{
typedef gcroot<HandWriting::MyControl^> MyControlItem;
typedef std::vector<MyControlItem> MyControlList;
typedef std::vector<std::pair<int, int>> LineControlSet;

private:

    unsigned lineConIndex;
    LineControlSet lineCon;
    MyControlItem myCon;

public:

    DistanceBetweenWrittenLine(HandWriting::MyControl^ mycon):
        lineConIndex(0), lineCon(), myCon(mycon)
    {
        int controlHeight = myCon->getMyControlHeight();
        int countRow = 0;
        int space = ConstantIdentifier::SPACE_CHAR;
        while (controlHeight - space > 0){
            controlHeight -= space;
            ++countRow;
        }
        int yUp = 0;
        int yUnder = 0;
        for (int i = 1; i < countRow; i *= 2){
            yUp += ConstantIdentifier::SPACE_CHAR;
            yUnder = yUp + ConstantIdentifier::SPACE_CHAR;
            lineCon.push_back(std::make_pair(yUp, yUnder));
            yUp = yUnder;
        }
    }
    std::pair<int, int> getLinePair();
    std::pair<int, int> getLinePair(unsigned index);
    void createNewLine();
    void setMyControl(HandWriting::MyControl^ mycon);
```

```

    unsigned getLineIndex() const;
    unsigned getLineCount();
    void increaseLineIndex();
    int getSpaceLine() const;
    void resetSpaceChar(int mul);
    void setSpaceLine(unsigned mul);
    void clearWritingPad();

```

```
};
```

```
virtual void increaseLineIndex() = 0;
```

เพิ่มจำนวนบรรทัด

```
virtual std::pair<int, int> getLinePair()= 0;
```

ให้ค่าแกน y ของเส้นบรรทัดคู่ที่ 1 โดยให้ค่าเป็น y_1, y_2

```
virtual std::pair<int, int> getLinePair(unsigned
index)= 0;
```

ให้ค่าแกน y ของเส้นบรรทัดคู่ใดๆตามที่กำหนดใน index โดยให้ค่าเป็น y_1, y_2

```
virtual void setMyControl(HandWriting::MyControl^ )
= 0;
```

สร้าง Association ไปยัง MyControl Instance ซึ่งเป็น object จัดการ Writing Pad

```
virtual void createNewLine() = 0;
```

สร้างเส้นบรรทัดคู่ใหม่

```
virtual unsigned getLineCount() = 0;
```

ให้ค่าจำนวนเส้นบรรทัดทั้งหมดที่มี

```
virtual int getSpaceLine() const = 0;
```

ให้ค่าความกว้างระหว่างเส้นบรรทัด

```
virtual void setSpaceLine(unsigned mul)= 0;
```

กำหนดค่าความกว้างระหว่างเส้นบรรทัด

```
virtual void clearWritingPad() = 0;
```

ลบลายมือเขียนบน Writing Pad

4.2.1.3. Class PrototypeControl คลาสจัดการกับ user interface เพื่อแสดงอักขรตัวอย่าง (prototype character) ที่มีลักษณะใกล้เคียงกับลายมือเขียนที่รับเข้าไปเป็นอินพุตมากที่สุด ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

public ref class PrototypeControl : public
System::Windows::Forms::UserControl
{
private:
    DisplayPrototype* disPtt;
    const MyPoint::PointList* prototypeLst;
public:
    void setDisplayPointList(const MyPoint::PointList* pntLst);
    void deleteDisplayPrototype();
    System::ComponentModel::Container ^components;
};

```

```

void setDisplayPointList
(const MyPoint::PointList);

```

กำหนดค่าให้โปรโตไทป์ลายมือเขียนที่ต้องการแสดงที่หน้าจอ Prototype Window

```

void deleteDisplayPrototype();

```

ลบโปรโตไทป์ลายมือเขียนที่แสดงที่หน้าจอ Prototype Window

4.2.1.4. Class WritingPad คลาสจัดการกับ user interface เพื่อแสดงอินพุตลายมือเขียน ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

public ref class MyControl : public
System::Windows::Forms::UserControl
{
private:
    System::ComponentModel::Container ^components;
    bool leftClicked;
    bool newLine;
    MyControlImpl* myImpl_;
    const MyPoint::PointList* writtenPntLst;
    CurveListBuilder* curveLB;
    LineAbstraction* disLine;
    SystemStateMachine::ProcessingMachine* processMan;
    SystemStateMachine::SystemStateStackInterface* stateStack;

    System::Windows::Forms::TextBox^ recogTextBox;
private:
    void adjustWritingArea(int x);
    void adjustWritingAreaVertical(int y);
    void myScroll(System::Object^ sender,
        System::Windows::Forms::ScrollEventArgs^ e);
    void myScrollValueChanged(System::Object^ sender,
        System::EventArgs^ e);
    void vScrollValueChanged(System::Object^ sender,
        System::EventArgs^ e);
public:
    void setProcessingMachine
        (SystemStateMachine::ProcessingMachine* processman);
    void setDistanceBetweenWrittenLine(LineAbstraction* line);
    void setCurveList(CurveListBuilder* curveLst);
    void setSystemStateStackInterface
        (SystemStateMachine::SystemStateStackInterface* stack);
    void clearWritingPad();
    int getMyControlHeight();

```

```
};
```

```
void setProcessingMachine  
(SystemStateMachine::ProcessingMachine*);
```

สร้าง Association ไปยัง ProcessingMachine Instance ซึ่งเป็น object ที่ควบคุมการประมวลผลโปรแกรมโดยรวม

```
void setDistanceBetweenWrittenLine  
(LineAbstraction*);
```

สร้าง Association ไปยัง LineAbstraction Instance ซึ่งเป็น object จัดการการตีเส้นบรรทัดบน Writing Pad

```
void setCurveList(CurveListBuilder* );
```

สร้าง Association ไปยัง CurveListBuilder Instance ซึ่งเป็น object จัดการและเก็บข้อมูลลายมือเขียนบน Writing Pad

```
void setSystemStateStackInterface  
(SystemStateMachine::SystemStateStackInterface* );
```

สร้าง Association ไปยัง SystemStateStack Instance ซึ่งเป็น object จัดการและเก็บข้อมูลตัวอักษรระหว่างการประมวลผล

```
void clearWritingPad();
```

ลบลายมือเขียนบน Writing Pad

4.2.1.5. Class GUIControl คลาสจัดการ user interface ทั้งหมดของหน้าต่างหลักของโปรแกรม ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
public ref class MyUI : public  
System::Windows::Forms::UserControl  
{  
    public:  
        property WriterManagement::WriterInterface* WriterIn {  
            WriterManagement::WriterInterface* get()  
            {  
                return writerIn;  
            }  
  
        void set(WriterManagement::WriterInterface* wrt)  
        {  
            writerIn = wrt;  
        }  
    }  
    private:  
        PrototypeManagement* prototypeMan;  
        UnicodeTransform::UnicodeToWChartMap* unicodeWchMap;  
        std::string* recogString;  
        CharPrototypeList* charVec;  
        CharNumPrototypeList* charNumVec;
```



```

RecognitionProcess::PrototypeList* prototypeList;

SystemStateMachine::ProcessingMachine* coreMan;
int mode;
WriterManagement::WriterInterface* writerIn;
LineAbstraction* disLine;
PrototypeControl^ protoControl;
Array^ disPrototypeLst;
public:
    int getScriptMode();
    System::Windows::Forms::TableLayoutPanel^
getMyControlPanel();
    System::Windows::Forms::TableLayoutPanel^
getPrototypeControlPanel();
    void setScriptTextBoxString(const std::string& str);
    void createDisPrototypeItem();
    char setDisplayTextBoxPrototype();
    void setPrototypeControl(PrototypeControl^ pttCon);
    void setRecogTextBoxString(std::string recText);
    void setProcessingMachine
(SystemStateMachine::ProcessingMachine* coreman);
    void
setWriterInterface(WriterManagement::WriterInterface*
writer);
    void setPrototypeList(RecognitionProcess::PrototypeList
pptLst);
    char setPointListPrototypeSelected(unsigned
pntLstDisplay);
    void setDistanceBetweenWrittenLine(LineAbstraction*
line);

};
int getScriptMode();

```

แสดงโหมดการประมวลผลที่ทำงานอยู่ในปัจจุบัน

```
void setScriptTextBoxString(const std::string&);
```

กำหนดข้อความที่จะแสดงออกที่ Script Window

```
void createDisPrototypeItem();
```

กำหนดตัวอักษรโปรโตไทป์ 5 ตัวที่จะแสดงที่ Text Box

```
char setDisplayTextBoxPrototype();
```

แสดงตัวอักษรโปรโตไทป์ทั้งหมด 5 ตัวอักษรที่ Text Box และให้ตัวอักษรโปรโตไทป์ที่มีลักษณะเหมือนอักษรลายมือเขียนมากที่สุดออกมา

```
void setPrototypeControl(PrototypeControl^);
```

สร้าง Association ไปยัง PrototypeControl Instance ซึ่งเป็น object แสดงตัวอักษรโปรโตไทป์

```
void setRecogTextBoxString(std::string);
```

กำหนดข้อความที่จะแสดงออกที่ Recognition Window

```
void setProcessingMachine
(SystemStateMachine::ProcessingMachine*);
```

สร้าง Association ไปยัง SystemStateStack Instance ซึ่งเป็น object จัดการและเก็บ
ข้อมูลตัวอักษรระหว่างการประมวลผล

```
void setWriterInterface
(WriterManagement::WriterInterface*);
```

สร้าง Association ไปยัง WriterInterface Instance ซึ่งเป็น object จัดการผู้ใช้ระบบ

```
void setPrototypeList
(RecognitionProcess::PrototypeList);
```

กำหนดค่า data member ชื่อ &prototypeMan->prototypeList

ให้มีค่าเท่ากับตัวอักษรโปรโตไทป์ทั้งหมด 5 ตัวที่ได้จากการรู้จำ

```
char setPointListPrototypeSelected
(unsigned index);
```

เลือกตัวอักษรโปรโตไทป์ที่จะแสดงที่ Prototype Window โดย index คือค่าที่ใช้อ้าง

ตำแหน่งในการเข้าถึงข้อมูลใน &prototypeMan->prototypeList

```
void setDistanceBetweenWrittenLine
(LineAbstraction* line);
```

สร้าง Association ไปยัง LineAbstraction Instance ซึ่งเป็น object จัดการการตีเส้น

บรรทัดบน Writing Pad

```
property WriterManagement::WriterInterface*
WriterIn {
    WriterManagement::WriterInterface* get()
    {
        return writerIn;
    }

    void set(WriterManagement::WriterInterface*
wrt)
    {
        writerIn = wrt;
    }
}
```

เป็นคุณสมบัติของ Class ซึ่งจัดการกับข้อมูลของผู้ใช้โปรแกรมขณะนั้น

4.2.1.6. Class CharacterDialogBox จัดการกับ user interface ของ Dialog box แสดง
ข้อมูลตัวอักษรและข้อมูลผู้ใช้จากฐานข้อมูล โดยคลาสประกอบด้วยฟังก์ชันต่าง ๆ ดังนี้

```
public ref class CharacterDialogBox : public
```

```

                                System::Windows::Forms::Form
{
private:
    DbUI^ dbUI;
    WriterUI^ wrtUI;
public:
    CharacterDialogBox(void)
    {
        InitializeComponent();
        dbUI = gcnew DbUI;
        dbUI->Dock =
            System::Windows::Forms::DockStyle::Fill;
        this->Controls->Add(dbUI);
    }

    CharacterDialogBox(const std::string& ui )
    {
        InitializeComponent();
        if (ui == ConstantIdentifier::CHARACTER){
            dbUI = gcnew DbUI;
            dbUI->Dock =
                System::Windows::Forms::DockStyle::Fill;

            this->Controls->Add(dbUI);
        }
        else if (ui == ConstantIdentifier::WRITER){
            wrtUI = gcnew WriterUI;
            wrtUI->Dock =
                System::Windows::Forms::DockStyle::Fill;

            this->Controls->Add(wrtUI);
        }
    }
};

```

4.2.1.7. Class DialogBoxView (class DbView) จัดการ user interface ในการ
แสดงผลลายมือเขียนที่โหลดมาจากฐานข้อมูล ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

public ref class DbView : public
System::Windows::Forms::UserControl
{
public:
    DbView(void): disPntLst(new DisplayPointList)
    {
        InitializeComponent();
        pointLst = &disPntLst->pointLst;
    }

private:
    DisplayPointList* disPntLst;
    MyPoint::PointList* pointLst;
    System::ComponentModel::Container ^components;
public:
    void setDisplayPointList(const std::string& pntStr);
};

void setDisplayPointList(const std::string&);

```

แสดงตัวอักษรลายมือเขียนในฐานข้อมูลทั้งหมด

4.2.1.8. Class DialogBoxUI จัดการฟังก์ชันการทำงานของ Control ต่างๆใน Dialog box ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
public ref class DbUI : public
System::Windows::Forms::UserControl
{
private:
    CharacterSetStruct* charSet;
    CharacterInterface::CharacterObjectSet* charInSet;
    UnicodeTransform::UnicodeToWChartMap* unicodewchMap;
    unsigned index;
    DbView^ dbView;
private:
    void exitButtonClick(System::Object^ sender,
        System::EventArgs^ e);
    void updateButtonClick(System::Object^ sender,
        System::EventArgs^ e);
    void deleteButtonClick(System::Object^ sender,
        System::EventArgs^ e);
    void loadButtonClick(System::Object^ sender,
        System::EventArgs^ e);
    void nextButtonClick(System::Object^ sender,
        System::EventArgs^ e);
    void setTextBoxOutput(unsigned curIndex );
    void setCharacterObjectValue
        (CharacterInterface::CharacterInterface& charIn);
    void setDisplayView();
    void setTextBoxOutNext();
    void setTextBoxOutBack();
    void backButtonClick(System::Object^ sender,
        System::EventArgs^ e);
};
```

void setTextBoxOutput(unsigned curIndex);

กำหนดข้อมูลตัวอักษรที่จะแสดงที่ Text Box

**void setCharacterObjectValue
(CharacterInterface::CharacterInterface& charIn);**

กำหนดข้อมูลตัวอักษรใหม่ที่จะแก้ไขในฐานข้อมูล

void setDisplayView();

แสดงตัวอักษรลายมือเขียนในฐานข้อมูลทั้งหมด

void setTextBoxOutNext();

กำหนดข้อมูลตัวอักษรที่จะแสดงที่ Text Box ตัวถัดไป

void setTextBoxOutBack();

กำหนดข้อมูลตัวอักษรที่จะแสดงที่ Text Box ตัวก่อนหน้า

4.2.2. ScriptManagement: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดอักษรต่างๆที่ต้องการให้ผู้ใช้งานระบบเขียนตามตัวพิมพ์

4.2.2.1. Class ScriptProcess คลาสจัดการกับตัวอักษรที่ต้องการให้ผู้ใช้งานระบบเขียนตาม ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class ScriptProcess
{
public:
    ScriptProcess();
    const std::string* beginWrdLst();
    const std::string* endWrdLst();
    WordLen getCurrCharPosition() const;
    ScriptWordListSize getCurrWordPosition() const;
    char nextChar();
    std::string getScriptString();
private:
    ScriptWordList scriptWordLst;
    ScriptWordListSize wordNum;
    ScriptWordListSize currWord;
    WordLen currCh;
};

char nextChar();
ไปยังตัวอักษรสกริปต์ตัวถัดไป

std::string getScriptString();
ให้สกริปต์กลับออกมา
```

4.2.3. RecognitionProcess: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดการติดต่อกับระบบของโครงงานเดิมเพื่อแปลงตัวอักษรลายมือเขียนเป็นอักษร

4.2.3.1. Class RecognitionInterface เป็น abstract class ซึ่งประกอบด้วย method ต่างๆ เพื่อติดต่อกับโครงงานเดิมในการแปลงอินพุทลายมือเขียนเป็นอักษรตัวพิมพ์ คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class RecognitionInterface
{
public:
    virtual void recognitionMain(const MyPoint::PointList&
                                pnt,PrototypeList& prototypeList, int yMinLevel,
                                int yMaxLevel) = 0;
};

virtual void recognitionMain
(const MyPoint::PointList& pnt,
PrototypeList& prototypeList,
int yMinLevel, int yMaxLevel) = 0;
```

จัดการการ Recognition โดยให้ผลกลับมาเป็นลำดับของอักษร โปรโทไทป์ที่ใกล้เคียงกับ
อักษรลายมือเขียนมากที่สุด 5 อันดับแรก

4.2.3.2. Class RecognitionProcess สืบทอด (inherit) มาจาก Class RecognitionInterface

คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class RecognitionProcess : public RecognitionInterface
{
public:
    RecognitionProcess();
    void recognitionMain(const MyPoint::PointList& pnt,
                        PrototypeList& prototypeList,
                        int yMinLevel, int yMaxLevel) ;
    char createSelectedPrototypeLst
        (ThaiRecogInterface* thaiRec,
         PrototypeList& prototypeList,
         std::string pntLstStr);
    char createNewPrototypeLst(const MyPoint::PointList&
                              wrt_pntLst , PrototypeList&
                              prototypeList, char c);
    virtual ~RecognitionProcess();
private:
    ThaiRecogInterface* inLineRecog;
    ThaiRecogInterface* upLineRecog;
    ThaiRecogInterface* underLineRecog;
};
```

```
char createSelectedPrototypeLst
    (ThaiRecogInterface* thaiRec,
     PrototypeList& prototypeList, std::string
     pntLstStr);
```

สร้าง prototypeList ซึ่งเป็นลำดับของอักษร โปรโทไทป์ 5 อันดับแรกที่มีลักษณะ
ใกล้เคียงกับอินพุตลายมือเขียนมากที่สุด

```
char createNewPrototypeLst(const
MyPoint::PointList&
    wrt_pntLst , PrototypeList&
    prototypeList, char c);
```

สร้าง prototypeList ของอักษรที่มีลักษณะใกล้เคียงกับอินพุตลายมือเขียนมากที่สุด

4.2.3.3. Class PrototypeCharacter เก็บข้อมูลอักษร โปรโทไทป์ที่ได้จากการรู้จำ

(Recognition) คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class PrototypeCharacter
{
public:
    PrototypeCharacter():pc_score(0), pc_char(0), p
                        c_charNum(0), pc_pntLst({})
    PrototypeCharacter(const PrototypePoint* begin, const
                        PrototypePoint* end, char ch, int num,
                        float score);
    PrototypeCharacter(const MyPoint::PointList& pnt,
                        char ch, int num);
```

```

        void setPrototypeCharPntLst(const PrototypePoint*
begin,
                                const PrototypePoint*
                                end);
        void setPrototypeChar(char ch);
        void setPrototypeCharNum(int num);
        void setPrototypeScore(float score);
        float getPrototypeScore() const;
        const MyPoint::PointList& getPrototypeCharPntLst()
const;
        char getPrototypeChar() const;
        int getPrototypeCharNum() const;
        const PrototypeCharacter& getPrototypeCharacter();
        void setPrototypeCharacter(const MyPoint::PointList&
pnt, char ch, int num);

private:
        float pc_score;
        char pc_char;
        int pc_charNum;
        MyPoint::PointList pc_pntLst;

};

```

4.2.4. SystemStateMachine: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดการกับตัวอักษรที่ไม่เป็นอักษรโคด เป็นส่วนหลักของระบบในการติดต่อกับเนมสเปซ (namespace) อื่นๆ คล้ายเป็น main function ของระบบ

4.2.4.1. Class RecogItem เป็นโครงสร้างข้อมูล โดยเก็บข้อมูลตัวอักษรโคด ซึ่งเป็นเอาท์พุทจากการรู้จำ (Recognition) ในแต่ละครั้ง ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

class RecogItem
{
private:
        int state;
        char recog;
        MyPoint::PointList pointList;
public:
        RecogItem(): state(ConstantIdentifier::s_init_state),
                    recog(0), pointList({})
        RecogItem(int st, char rec = 0, const
MyPoint::PointList&
                    pntLst = MyPoint::PointList());
        void setState(int st);
        void setRecogChar(char rec);
        void setWrittenPntLst(const MyPoint::PointList&
pntLst);
        int getState() const;
        char getRecogChar() const;
        MyPoint::PointList getWrittenPntLst()const;
};

```

void setState(int st);

กำหนดState ID

void setRecogChar(char rec);

กำหนดอักษรโคด ที่ได้จากการ Recognition

```
void setWrittenPntLst
(const MyPoint::PointList&);
```

กำหนดลายมือเขียนของอักขรโคด

```
int getState() const;
```

ให้ State ID

```
char getRecogChar() const;
```

ให้ อักขรโคด ที่ได้จากการ Recognition

```
MyPoint::PointList getWrittenPntLst()const;
```

ให้อักขรโคด ที่ได้จากการ Recognition

4.2.4.2. Class RecogList เนื่องจากตัวอักขรบางตัวไม่เป็นอักขรโคด ซึ่งต้องผ่านการประมวลผลหลายครั้ง ทำให้ได้ instance ของ Class RecogItem มากกว่า 1 instance คลาสนี้จึงสร้างขึ้นเพื่อเก็บ instance เหล่านั้น

4.2.4.3 Class StateContent เนื่องจากโปรแกรมจัดการกับอักขรที่ไม่เป็นอักขรโคดโดยใช้หลักการของ State Machine อักขรแต่ละตัวจึงถูกเก็บไว้ใน State คลาสนี้แสดงฟังก์ชันต่างๆในการจัดการกับแต่ละ State

```
class StateContent
{
public:
    StateContent():script(0), scriptLabel(0), recogRes(0),
        strokeCountConstant(ConstantIdentifier::IGNORE_STROKE),
        strokeCount(0), recogList({})
    StateContent(const RecogItem& item);
    void addRecogItem(const RecogItem& item);
    void addRecogItem(int st, char rec = 0,
        const MyPoint::PointList& pntLst =
        MyPoint::PointList());

    void setScriptChar(char ch);
    void setScriptLabel(char ch);
    void setRecogResult(char ch);
    void setStrokeCountConstant(unsigned stroke);
    void setStrokeCount(unsigned stroke);
    void increaseStrokeCount();

    char getRecogResult() const;
    char getScriptChar() const;
    char getScriptLabel() const;
    unsigned getStrokeCount() const;
    unsigned getStrokeCountConstant() const;
    int getState() const;
    MyPoint::PointList getPointList();
    MyPoint::PointList getPrevLastRecogItemPntLst();
    const RecogList& getRecogLst() const;
    RecogListSize getRecItemNumRemove();
```



```

        RecogItem getLastRecogItem();

        int deleteRecogItem(RecogList::iterator pos);
        bool isRecogListEmpty() const;
        RecogList::iterator endRecogList();
        RecogListSize recogItemAmount();
        bool isRecogCharLikeScript();
    private:
        char script;
        char scriptLabel;
        char recogRes;
        unsigned strokeCountConstant;
        unsigned strokeCount;
        RecogList recogList;
};

```

void addRecogItem(const RecogItem& item);

กำหนดข้อมูลอักษรที่ได้หลังจากการ Recognition ลง State

```

void addRecogItem(int st, char rec = 0,
const MyPoint::PointList& pntLst =
MyPoint::PointList());

```

กำหนดข้อมูลอักษรที่ได้หลังจากการ Recognition ลง State

void setScriptChar(char ch);

กำหนดตัวอักษรสคริปต์ของ State

void setScriptLabel(char ch);

กำหนดตัวอักษรสคริปต์ของ State

void setRecogResult(char ch);

กำหนดตัวอักษรเอาท์พุทของ State

void setStrokeCountConstant(unsigned stroke);

กำหนด Stroke การเขียนตัวอักษรของ State

void increaseStrokeCount()

เพิ่มจำนวน Stroke การเขียนตัวอักษรของ State

char getRecogResult() const;

ให้ตัวอักษรเอาท์พุทของ State

char getScriptChar() const;

ให้ตัวอักษรสคริปต์ของ State

char getScriptLabel() const;

ให้ตัวอักษรสคริปต์ของ State

unsigned *getStrokeCount()* *const*;

ให้ Stroke การเขียนตัวอักษรว่าเขียนไป Stroke ที่เท่าใดแล้ว

unsigned *getStrokeCountConstant()* *const*;

ให้ Stroke การเขียนตัวอักษรของ State

int *getState()* *const*;

ให้ State ID

MyPoint::PointList *getPointList()*;

ให้ลายมือเขียนตัวอักษรใด

bool *isRecogListEmpty()* *const*;

ตรวจสอบว่ามีข้อมูลอยู่ใน State หรือไม่

4.2.4.4. Class SystemStackStateInterface เป็น abstract class ซึ่งประกอบด้วยฟังก์ชันต่างๆในการจัดการ Stack ซึ่งเก็บ State ต่างๆ ของตัวอักษรไว้ คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class SystemStateStackInterface
{
public:
    virtual void pushRecogModeState() = 0;
    virtual void pushScriptModeState(char ch_script) = 0;
    virtual void pushRecState(int state, char ch_reg,
        char ch_output, const MyPoint::PointList& pntLst)
    = 0;
    virtual void addRecogList(int state, char ch_reg = 0,
    char ch_outPut = 0, const MyPoint::PointList& pntLst =
    MyPoint::PointList()) = 0;
    virtual void addRecogList(int state, char ch_reg,
        const MyPoint::PointList& pntLst) = 0;

    virtual const StateContent& getPrevCurStateContent() =
    0;

    virtual int getCurrentState() = 0;
    virtual char getCurrRecogChar() = 0;
    virtual char getCurrScriptChar() = 0;
    virtual char getCurrScriptLabel() = 0;
    virtual char getCurrCharFromRecLst() = 0;
    virtual unsigned getCurrStrokeCount() = 0;
    virtual unsigned getCurrStrokeCountConstant() = 0;
    virtual MyPoint::PointList getPrevCurrPointList() = 0;
    // virtual MyPoint::PointList getCurrPointList() = 0;
    virtual std::string getOutputRecogText() = 0;
    virtual int getStateNumber() = 0;

    virtual bool isEmpty() = 0;
    virtual bool isCurrRecogedListEmpty() = 0;
    virtual bool isFinishScriptState() = 0;
    virtual void increaseCurrStrokeCount() = 0;
    virtual void resetScriptChar(char ch) = 0;
    virtual void clearStack() = 0;
```

```

        virtual void setCurrScriptLabel(char ch) = 0;
        virtual void setWriterInterface
        (WriterManagement::WriterInterface* writer) = 0;

        virtual void saveStateChangeRecogChar(char ch = 0) = 0;
        virtual void saveState() = 0;
        virtual void popStack() = 0;

};

```

```

virtual void pushRecogModeState() = 0;

```

เพิ่ม Recognition State เข้า State Stack

```

virtual void pushScriptModeState(char ch_script) =
0;

```

เพิ่ม Script State เข้า State Stack

```

virtual void pushRecState(int, char ,char,
const MyPoint::PointList&) = 0;

```

เพิ่ม Recognition State เข้า State Stack

```

virtual void addRecogList (int state,
char ch_reg = 0, char ch_outPut = 0,
const MyPoint::PointList& pntLst =
MyPoint::PointList()) = 0;

```

เพิ่มผลการ Recognition อักษร โคด เข้า State

```

virtual void addRecogList(int state, char ch_reg,
const MyPoint::PointList& pntLst) = 0;

```

เพิ่มผลการ Recognition อักษร โคด เข้า State

```

virtual int getCurrentState() = 0;

```

ให้ State ปัจจุบันใน Stack

```

virtual char getCurrRecogChar() = 0;

```

ให้ อักษร โคด ที่ Reconition ได้ล่าสุดใน Stack

```

virtual char getCurrScriptChar() = 0;

```

ให้ อักษรสคริปต์ล่าสุดใน Stack

```

virtual char getCurrScriptLabel() = 0;

```

ให้ อักษรสคริปต์ล่าสุดใน Stack

```

virtual unsigned getCurrStrokeCount() = 0;

```

ให้ Stroke การเขียนตัวอักษรว่าเขียนไป Stroke ที่เท่าใดแล้ว

```

virtual unsigned getCurrStrokeCountConstant() = 0;

```

ให้ Stroke การเขียนตัวอักษรของ State

```
virtual MyPoint::PointList getCurrPointList() = 0;
```

ให้ อักษรลายมือเขียนล่าสุดใน Stack

```
virtual bool isEmpty() = 0;
```

ตรวจสอบว่า เป็น Stack เปล่าหรือไม่

```
virtual bool isCurrRecogedListEmpty() = 0;
```

ตรวจสอบว่ามีข้อมูลผลการ Recognition อยู่ใน State หรือไม่

```
virtual std::string getOutputRecogText() = 0;
```

ให้ String เอาท์พุท

```
virtual void clearStack() = 0;
```

ลบ State ทั้งหมดใน Stack

```
virtual void increaseCurrStrokeCount() = 0;
```

เพิ่มค่า Stroke การเขียนตัวอักษร

```
virtual int getStateNumber() = 0;
```

ให้ State ID ได้ล่าสุดใน Stack

```
virtual void setCurrScriptLabel(char ch) = 0;
```

กำหนดตัวอักษรสคริปต์ของ State

```
virtual void setWriterInterface  
(WriterManagement::WriterInterface* writer) = 0;
```

กำหนด ความสัมพันธ์ไปยัง WriterInterface ซึ่งจัดการข้อมูลของผู้เขียน

```
virtual void saveStateChangeRecogChar(char ch = 0)  
= 0;
```

แก้ไขข้อมูลในฐานข้อมูลและบันทึกการแก้ไขนั้น

```
virtual void saveState() = 0;
```

บันทึกข้อมูลลงฐานข้อมูล

```
virtual void popStack() = 0;
```

ลบ State ล่าสุดออกจาก Stack

4.2.4.5. Class SystemStackState สืบทอด (inherit) มาจาก Class

SystemStackStateInterface คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class SystemStateStack :public SystemStateStackInterface
{
public:
    SystemStateStack();
    void pushRecogModeState();
    void pushScriptModeState(char ch_script);
    void pushRecState(int state, char ch_reg, char
ch_output,
        const MyPoint::PointList& pntLst);
    void reduceStateNumber();
    void addRecogList(int state, char ch_reg = 0,
char ch_outPut = 0, const MyPoint::PointList&
pntLst = MyPoint::PointList());
    void addRecogList(int state, char ch_reg, const
MyPoint::PointList& pntLst);

    const StateContent& getPrevCurStateContent();
    int getCurrentState();
    char getCurrRecogChar();
    char getCurrScriptChar();
    char getCurrScriptLabel();
    char getCurrCharFromRecLst();
    unsigned getCurrStrokeCount();
    unsigned getCurrStrokeCountConstant();

    char getTrickCurScriptChar();

    MyPoint::PointList getCurrPointList();
    bool isEmpty();
    bool isCurrRecogedListEmpty();
    std::string getOutputRecogText();
    void resetStateValue(int st, char recogRes);
    void resetScriptChar(char ch);
    void clearStack();
    MyPoint::PointList getPrevCurrPointList();

    void increaseCurrStrokeCount();
    bool isFinishScriptState();
    int getStateNumber();
    void setCurrScriptLabel(char ch);
    void setCurrRecogLabel(char ch);

    void
setWriterInterface(WriterManagement::WriterInterface*
writer);
    void saveStateToDB(char recCH, const
MyPoint::PointList&
pntLst);
    void saveStateChangeRecogChar(char ch = 0);
    void saveState();
    void popStack();
private:
    unsigned charID;
    WriterManagement::WriterInterface* writerIn;
    std::string outputTextRecog;
    StateStack stateStack;
    ScriptStrokeCountMap scriptStrokeMap;
    ASCIIIScriptCharMap asciiScpMap;
};
```

4.2.4.6. Class ProcessingMachine ดูแลและควบคุมขั้นตอนการประมวลผลโปรแกรมทั้งหมด ทำหน้าที่คล้าย main function ของโปรแกรม คลาสประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class ProcessingMachine
{
public:
    ProcessingMachine();
    void setRunningMode(int rmode);

    void setMyUI(HandWriting::MyUI^ myui);
    void setSystemStateStack(SystemStateStackInterface*
stack);
    void setCurveListBuilder(CurveListBuilder* curveLstBD);
    void setRecognitionInterface
(RecognitionProcess::RecognitionInterface* recog);
    void setScriptProcess
(ScriptManagement::ScriptProcess* scptMan);

    void machineMainRoutine(int yMinLevel, int yMaxLevel);

    void coreProcess(int yMinLevel, int yMaxLevel);

    void machineTranspose(const MyPoint::PointList&
inputPntLst, char input);
    std::string activatedScriptMode();
    std::string pushScriptState();
    void setSelectedPrototypeChar(char ch);
    void setMyUIRecogTextBox();
    void clearStack();
private:
    int mode;
    unsigned selectedPrototypeChar;
    gcroot<HandWriting::MyUI^> myUI;
    RecognitionProcess::RecognitionInterface* recogMan;
    ScriptManagement::ScriptProcess* scriptMan;
    SystemStateStackInterface* stateStack;
    CurveListBuilder* curveLst;
    RecognitionProcess::PrototypeList prototypeList;
    MachineStateMap machineMap;
    ScriptCharMap scriptCharMap;
};
```

void setRunningMode(int rmode);

กำหนดโหมดการประมวลผลโปรแกรม

void setMyUI(HandWriting::MyUI^ myui);

กำหนด ความสัมพันธ์ไปยัง MyUI Instant

**void setSystemStateStack
(SystemStateStackInterface* stack);**

กำหนด ความสัมพันธ์ไปยัง SystemStateStack Instant

```
void setCurveListBuilder
(CurveListBuilder* curveLstBD);
```

กำหนด ความสัมพันธ์ไปยัง CurveListBuilder Instant

```
void setRecognitionInterface
(RecognitionProcess::RecognitionInterface*);
```

กำหนด ความสัมพันธ์ไปยัง Recognition Instant

```
void setScriptProcess
(ScriptManagement::ScriptProcess* scptMan);
```

กำหนด ความสัมพันธ์ไปยัง Script Instant

```
void machineMainRoutine(int yMinLevel, int
yMaxLevel);
```

คล้าย main function ดูผลการประมวลผลโปรแกรมโดยรวม

```
void coreProcess(int yMinLevel, int yMaxLevel);
```

ฟังก์ชันดูผลการประมวลผลโปรแกรมส่วนหาอักษรโดดจากการ Recognition

```
void machineTranspose(const MyPoint::PointList&
inputPntLst, char input);
```

ฟังก์ชันดูผลการประมวลผลโปรแกรมส่วนหาอักษรเอทพุทโดยอาศัย State Machine

```
std::string activatedScriptMode();
```

ฟังก์ชันสั่งเริ่มการประมวลผลโปรแกรมแบบโหมดสคริปต์ และให้ Script String
ออกมา

```
void cleanWrittingPad(const RemovedStatedMessage&
removedMss);
```

ลบลายมือเขียนบน Writing Pad

```
std::string pushScriptState();
```

เพิ่ม Script State เข้า State Stack และให้ Script String ออกมา

```
void setSelectedPrototypeChar(char ch);
```

กำหนดอักษรโปรโตไทป์ที่ถือว่ามีความลักษณะใกล้เคียงลายมือเขียนมากที่สุด

```
void setMyUIRecogTextBox();
```

กำหนด String ที่จะแสดงออกที่ Recognition Window

```
void clearStack();
```

ลบ State ทั้งหมดใน Stack

4.2.4.7. Class PrototypeCharacter คลาสซึ่งระบุและแสดงโครงสร้างข้อมูล โดยเก็บข้อมูลตัวอักษรเอาต์พุต 5 ตัวที่ได้มาจากการเรียกโครงการเดิมในแต่ละครั้ง

4.2.5. WriterManagement: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดการเกี่ยวกับผู้ใช้งานระบบ โดยหากมีผู้ใช้งานระบบใหม่เข้ามาจะเก็บข้อมูลของผู้ใช้งานระบบเข้าสู่ฐานข้อมูล

4.2.5.1. Class WriterInterface เป็น abstract class ซึ่งประกอบด้วยฟังก์ชันต่างๆ ในการจัดการกับข้อมูลของผู้ใช้ระบบ (Writer) ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class WriterInterface
{
public:
    virtual unsigned saveWriter() = 0;
    virtual const DBManagement::Writer& getWriter()
        const = 0;
    virtual void deleteWriterFromDB(const char* db, const
        char* host, const char* usr) = 0;
    virtual void updateWriterTODB(const char* db, const
        char* host, const char* usr) =
        0;
    virtual void updateTimeWritten (WriterInterface*
        wrtIn, const char* db,
        const char* host, const
        char* usr) = 0;
    virtual void updateTimeWritten (const char* db,
        const
        char* host, const char*
        usr) = 0;
    virtual void updateWriterNameAndStdID(const
        std::string&
        name, const std::string& stdID,
        const char* db, const char*
        host, const char* usr) = 0;

    virtual void setWriterID(unsigned id) = 0;
    virtual void setName(const std::string& nm) = 0;
    virtual void setFirstWrtTime
        (const std::string& fstTime) = 0;
    virtual void setLastWrtTime
        (const std::string& lstTime) = 0;
    virtual void setStudentID
        (const std::string& stdID) = 0;

    virtual void setName(System::String^ nm) = 0;
    virtual void setFirstWrtTime
        (System::String^ fstTime) = 0;
    virtual void setLastWrtTime
        (System::String^ lstTime) = 0;
    virtual void setStudentID(System::String^ stdID) = 0;

    virtual unsigned getWriterID() const = 0;
    virtual const std::string& getName() const = 0;
    virtual const std::string& getFirstWrtTime() const = 0;
    virtual const std::string& getLastWrtTime() const = 0;
    virtual const std::string& getStudentID() const = 0;
```



```

virtual System::String^ getSysStringWriterID() = 0;
virtual System::String^ getSysStringName() = 0;
virtual System::String^ getSysStringFirstWrtTime() = 0;
virtual System::String^ getSysStringLastWrtTime() = 0;
virtual System::String^ getSysStringStudentID() = 0;

};

```

```
virtual unsigned saveWriter() = 0;
```

เก็บข้อมูลผู้ใช้ระบบลงฐานข้อมูล

```
virtual const DBManagement::Writer& getWriter()
const = 0;
```

ให้ข้อมูลของผู้ใช้ระบบในปัจจุบัน

```
virtual void deleteWriterFromDB(const char* db,
const char* host, const char* usr) = 0;
```

ลบเก็บข้อมูลผู้ใช้ระบบจากฐานข้อมูล

```
virtual void updateWriterTODB(const char* db, const
char* host, const char* usr) = 0;
```

แก้ไขข้อมูลผู้ใช้ระบบในฐานข้อมูล

```
virtual void updateLastTimeWritten
(WriterInterface* wrtIn, const char* db, const
char* host, const char* usr) = 0;
```

แก้ไขข้อมูลเวลาการใช้ระบบครั้งล่าสุดของผู้ใช้ในฐานข้อมูล

```
virtual void updateLastTimeWritten (const char* db,
const char* host, const char* usr) = 0;
```

แก้ไขข้อมูลเวลาการใช้ระบบครั้งล่าสุดของผู้ใช้ในฐานข้อมูล

```
virtual void updateWriterNameAndStdID(const
std::string& name, const std::string& stdId,
const char* db, const char* host, const
char* usr) = 0;
```

แก้ไขข้อมูลชื่อและรหัสนักศึกษาของผู้ใช้ระบบในฐานข้อมูล

4.2.5.2. Class WriterProfile สืบทอด (inherit) มาจาก Class WriterInterface คลาส

ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

class WriterProfile : public WriterInterface
{
public:
    WriterProfile();
    WriterProfile(const std::string& fstWrtTime,
const std::string& lstWrtTime,
const std::string& stdID, const std::string&
nme);
    WriterProfile(const std::string& nme,

```

```

        const std::string& stdID);

WriterProfile(const DBManagement::Writer& writer);
WriterProfile(const WriterProfile& writer);
WriterProfile& operator=
    (const DBManagement::Writer& writer);
unsigned saveWriter();
const DBManagement::Writer& getWriter() const;
void deleteWriterFromDB(const char* db,
    const char* host, const char* usr);
void updateWriterTODB(const char* db,
    const char* host, const char* usr);
void updateLastTimeWritten (WriterInterface* wrtIn,
    const char* db, const char* host, const char*
usr);
void updateLastTimeWritten (const char* db,
    const char* host, const char* usr);
void updateWriterNameAndStdID(const std::string& name,
    const std::string& stdId, const char* db, const
char* host, const char* usr);

void setWriterID(unsigned id);
void setName(const std::string& nm);
void setFirstWrtTime(const std::string& fstTime);
void setLastWrtTime(const std::string& lstTime);
void setStudentID(const std::string& stdID);

void setName(System::String^ nm);
void setFirstWrtTime(System::String^ fstTime);
void setLastWrtTime(System::String^ lstTime);
void setStudentID(System::String^ stdID);

unsigned getWriterID() const;
const std::string& getName() const;
const std::string& getFirstWrtTime() const;
const std::string& getLastWrtTime() const;
const std::string& getStudentID() const;

System::String^ getSysStringWriterID();
System::String^ getSysStringName();
System::String^ getSysStringFirstWrtTime();
System::String^ getSysStringLastWrtTime();
System::String^ getSysStringStudentID();

private:
    unsigned writerID;
    std::string firstWrtTime;
    std::string lastWrtTime;
    std::string studentID;
    std::string name;
};

```

4.2.6. CharacterInterface: แหล่งรวมของโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่เป็นสื่อกลางในการติดต่อกันระหว่างข้อมูลจากฐานข้อมูลและ การแสดงผลข้อมูลออกทางหน้าจอ

4.2.6.1. Class CharacterInterface ประกอบด้วยฟังก์ชันในการติดต่อกับ CharacterDB

Instant เพื่อแปลงอักขรระหว่างรหัส ASCII และ รหัส UNICODE ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

class CharacterInterface
{
public:
    CharacterInterface();
    CharacterInterface
        (const DBManagement::CharacterDB& chDB);
    CharacterInterface& operator=
        (const DBManagement::CharacterDB& chDB);

    void setCharID(const System::String^ chId);
    void setWriterID(const System::String^ writerId);
    void setScriptChar(const System::String^ scpCh);
    void setRecogChar(const System::String^ recCh);
    void setEditChar(const System::String^ edCh);
    void setWriteTime(const System::String^ wrtTime);
    void setWrtPointList(const System::String^ pntLst);
    void setEditPointList(const System::String^ pntLst);
    void setStartPoint(const System::String^ strPnt);
    void setEndPoint(const System::String^ nPnt);

    void setCharID(const std::string& chId);
    void setWriterID(const std::string& writerId);
    void setScriptChar(const std::string& scpCh);
    void setRecogChar(const std::string& recCh);
    void setEditChar(const std::string& edCh);
    void setWriteTime(const std::string& wrtTime);
    void setWrtPointList(const std::string& pntLst);
    void setEditPointList(const std::string& pntLst);
    void setStartPoint(const std::string& strPnt);
    void setEndPoint(const std::string& nPnt);

    System::String^ getCharID();
    System::String^ getWriterID();
    System::String^ getScriptChar();
    System::String^ getRecogChar();
    System::String^ getEditChar();
    System::String^ getWriteTime();
    System::String^ getWrtPointList();
    System::String^ getEditPointList();
    System::String^ getStartPoint();
    System::String^ getEndPoint();

    unsigned getUnsignedCharID() const;
    unsigned getUnsignedWriterID() const;
    const std::string& getStringScriptCh() const;
    const std::string& getStringRecogCh() const;
    const std::string& getStringEditCh() const;
    const std::string& getStringWriteTime() const;
    const std::string& getStringWrtPntLst() const;
    const std::string& getStringEditPntLst() const;
    int getIntStartPoint() const;
    int getIntEndPoint() const;

    void deleteCharFromDB(const char* db, const char*
        host, const char* usr);
    void updateCharTODB(const char* db, const char*
        host, const char* usr);
private:

```

```

        unsigned charID;
        unsigned writerID;
        std::string scriptCh;
        std::string recogCh;
        std::string editCh;
        std::string writeTime;
        std::string wrtPntLst;
        std::string editPntLst;
        int startPnt;
        int endPnt;
};

```

```

void deleteCharFromDB(const char* db, const char*
host, const char* usr);

```

ลบข้อมูลตัวอักษรจากฐานข้อมูล

```

void updateCharTODB(const char* db, const char*
host, const char* usr);

```

แก้ไขข้อมูลตัวอักษรในฐานข้อมูล

4.2.6.2. Class CharacterObjectSet จัดการกับลำดับของ CharacterInterface Instance

4.2.7. DBManagement: รวบรวมโมดูล คลาส และ ฟังก์ชันต่างๆ ที่ทำหน้าที่จัดการติดต่อกับฐานข้อมูล

4.2.7.1. Class CharacterDB เก็บข้อมูลของตัวอักษรต่างๆ ที่ไหลมาจากฐานข้อมูล ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

class CharacterDB
{
private:
    typedef mysqlpp::Null< int , mysqlpp::NullisZero >
                                                IndexPntList;

    Writer writer;
    unsigned int charID;
    unsigned int writerID;
    MyString writeLabel;
    MyString recogLabel;
    MyString editLabel;
    mysqlpp::DateTime writtenTime;
    std::string writtenPointList;
    MyString editPointList;
    IndexPntList startPoint;
    IndexPntList endPoint;
public:
    CharacterDB(): charID(0),writerID(0),writeLabel(),
        recogLabel(),editLabel(),writtenTime(),
        writtenPointList(),editPointList(),
        startPoint(0),endPoint(0){}
    CharacterDB(const Writer& wrt, std::string wrtLB,
        std::string regLB, const std::string&
        wrtPntList, const char* db, const char*
        host, const char* usr, const char* pass="",

```

```

        std::string editLB="", std::string
        editPntLst="",int start=0, int end=0);

CharacterDB(unsigned wrtID, std::string wrtLB,
        std::string regLB, const std::string&
        wrtPntList, const char* db, const char*
        host, const char* usr, const char* pass="",
        std::string editLB="", std::string
        editPntLst="",int start=0, int end=0);

CharacterDB(const mysqlpp::Row& row):
        charID(row[unsigned(0)]),
        writerID(row[1]),
        writeLabel(std::string(row[2])),
        recogLabel(std::string(row[3])),
        editLabel(std::string(row[4])),
        writtenTime(row[5]),
        writtenPointList(row[6]),
        editPointList(std::string(row[7])),
        startPoint(int (row[8])),
        endPoint(int (row[9])) {}

CharacterDB& operator=(const mysqlpp::Row& row)
{
        charID = row[unsigned(0)];
        writerID = row[1];
        writeLabel = std::string(row[2]);
        recogLabel = std::string(row[3]);
        editLabel = std::string(row[4]);
        writtenTime = row[5];
        writtenPointList = row[6];
        editPointList = std::string(row[7]);
        startPoint = int (row[8]);
        endPoint = int (row[9]);
        return *this;
}

CharacterDB
(const CharacterInterface::CharacterInterface& chIn);
CharacterDB& operator=
(const CharacterInterface::CharacterInterface& chIn);

mysqlpp::SQLString getCharID() const;
mysqlpp::SQLString getWriterID() const;
mysqlpp::SQLString getWrtLabel() const;
mysqlpp::SQLString getRecogLabel() const;
mysqlpp::SQLString getEditLabel() const;
mysqlpp::SQLString getWrtTime() const;
mysqlpp::SQLString getWrtPntList() const;
mysqlpp::SQLString getEditPntList() const;
mysqlpp::SQLString getStartPnt() const;
mysqlpp::SQLString getEndPnt() const;

unsigned getCurrWrtCharID() const;

void updateCharacter
        const CharacterInterface::CharacterInterface&
        chIn,const char* db, const char* host,
        const char* usr, const char* pass = "");

void updateRecogChar(unsigned charId, std::string

```

```

        recogChar, const char* db, const char* host, const
        char* usr, const char* pass = "");
void deleteRecCharRow(unsigned charId, const char* db,
        const char* host, const char* usr,
        const char* pass = "");
unsigned getCharIDUnsigned() const;
unsigned getWriterIDUnsigned() const;
std::string getScriptLabelString() const;
std::string getRecogLabelString() const;
std::string getEditLabelString() const;
std::string getWrtTimeString() const;
std::string getWrtPntListString() const;
std::string getEditPntListString() const;
int getStartPntInt() const;
int getEndPntInt() const;
};

```

```

void updateCharacter(const
CharacterInterface::CharacterInterface& chIn, const
char* db, const char* host, const char* usr, const
char* pass = "");

```

แก้ไขข้อมูลตัวอักษรในฐานข้อมูล

```

void updateRecogChar(unsigned charId, std::string
recogChar,
const char* db, const char* host, const char* usr,
const char* pass = "");

```

แก้ไขผลการ Recognition ของลายมือเขียนในฐานข้อมูล

```

void deleteRecCharRow(unsigned charId,
const char* db, const char* host, const char* usr,
const char* pass = "");

```

ลบข้อมูลตัวอักษรในฐานข้อมูล

4.2.7.2. Class Writer เก็บข้อมูลของผู้เขียนหรือผู้ใช้ระบบซึ่งโหลดจากฐานข้อมูล

ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```

class Writer
{
private:
    typedef mysqlpp::Null<mysqlpp::DateTime,
        mysqlpp::NullisBlank> WrittenTime;
    unsigned int writerID;
    WrittenTime firstWrtTime;
    WrittenTime lastWrtTime;
    MyString studentID;
    MyString name;
public:
    Writer():writerID(0), firstWrtTime(), lastWrtTime(),
        studentID(), name(){}
    Writer(const mysqlpp::Row& row):
        writerID(row[unsigned (0)]),
        firstWrtTime(mysqlpp::DateTime(row[1])),
        lastWrtTime(mysqlpp::DateTime (row[2])),

```

```

        studentID(std::string(row[3])),
        name(std::string (row[4])){}
Writer(const char* db, const char* host,
const char* usr, const char* pass="",
std::string stdID="");
Writer& operator=(const mysqlpp::Row& row)
{
    writerID = row[unsigned(0)];
    firstWrtTime = std::string(row[1]);
    lastWrtTime = std::string(row[2]);
    studentID = std::string(row[3]);
    name = std::string(row[4]);
    return *this;
}
Writer(const WriterManagement::WriterInterface& wrtIn);
Writer& operator=
    (const WriterManagement::WriterInterface& wrtIn);

mysqlpp::SQLString getStringWriterID() const;
mysqlpp::SQLString getWriterName
    (const char* db, const char* host,
const char* usr, const char* pass="") const;
mysqlpp::SQLString getWriterName() const;
mysqlpp::SQLString getFstWrtTime() const;
mysqlpp::SQLString getLstWrtTime() const;
mysqlpp::SQLString getStdID() const;

unsigned    getWriterID() const;
std::string getStringWriterName() const;
std::string getStringFstWrtTime() const;
std::string getStringLstWrtTime() const;
std::string getStringStdID() const;

void setWriterID(unsigned wrtID);
void setName(const std::string& nm );
void setFirstWritingTime(const std::string& fstTime);
void setLastWritingTime(const std::string& lstTime);
void setStudentID(const std::string& stdID);

unsigned insertWriter
    (const char* db, const char* host,const char*
usr);
void updateLastWrtTime
    (const std::string& userName, const char* db,
const char* host,const char* usr, const char*
pass="");
unsigned int getMaxWriterID
    (const char* db, const char* host,const char*
usr,
const char* pass="")const;
void updateWriter
    (const WriterManagement::WriterInterface& wrtIn,
const char* db,const char* host,const char* usr,
const char* pass = "");
void deleteWriter
    (const WriterManagement::WriterInterface& wrtIn,
const char* db,const char* host,const char* usr,
const char* pass = "");
void updateWriterNameAndStdID
    (unsigned wrtID, const std::string& name,
const std::string& stdId,const char* db,
const char* host,const char* usr);

```

```
};
```

```
unsigned insertWriter(const char* db, const char*
host, const char* usr);
```

เพิ่มผู้ใช้ระบบเข้าฐานข้อมูล

```
void updateLastWrtTime(const std::string& userName,
const char* db, const char* host, const char* usr,
const char* pass="");
```

แก้ไขเวลาการเข้าใช้งานครั้งล่าสุดของผู้ใช้ในฐานข้อมูล

```
unsigned int getMaxWriterID(const char* db, const
char* host, const char* usr, const char*
pass="")const;
```

ให้รหัส Writer ID สูงสุดที่มีในฐานข้อมูล

```
void updateWriter(const
WriterManagement::WriterInterface& wrtIn, const
char* db, const char* host, const char* usr, const
char* pass = "");
```

แก้ไขข้อมูลของผู้ใช้ในฐานข้อมูล

```
void deleteWriter(const
WriterManagement::WriterInterface& wrtIn, const
char* db, const char* host, const char* usr, const
char* pass = "");
```

ลบข้อมูลของผู้ใช้ในฐานข้อมูล

```
void updateWriterNameAndStdID(unsigned wrtID, const
std::string& name, const std::string& stdId, const
char* db, const char* host, const char* usr);
```

แก้ไขข้อมูลชื่อ และรหัสนักศึกษาของผู้ใช้ในฐานข้อมูล

4.2.7.3. Class DBConnection ติดต่อฐานข้อมูล ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

```
class DBConnection
{
private:
    const char* dbName;
    const char* hostAddress;
    const char* userName;
    const char* passWord;
    mysqlpp::Connection con;

public:
    DBConnection():con(){}
    DBConnection(const char* db, const char* host = "",
const char* user= "", const char*
```



```

        password=""):dbName(db),hostAddress(host),
        userName(user),password(password),con(db,
        host, user, password)
    {
        mysqlpp::Query query = con.query();
        std::string setName = "set names tis620";
        query.exec(setName);
        query.reset();
    }

    bool isWriterExist(std::string name);
    bool isThereAnyWriter();

    bool selectWriterByRow(std::string name,Writer& wrt);
    bool selectWriterByRow(std::string name);
    unsigned int selectWriterID(std::string name);
    std::string selectWrittenTime(std::string wrtTime,
                                unsigned int wrt_id);

    bool insertWriter(const Writer& wrt);
    unsigned int insertCharacter(const CharacterDB& chDB );
    unsigned int insertNewWriter(const Writer& wrt);

    void updateLstWrtTime(unsigned int wrtID,
                        std::string lstTime);
    void updateRecogLabel(unsigned int charId,
                        std::string recogChar);
    void updateCharacter(const CharacterDB& chDB);
    void updateWriter(const Writer& wrt);
    void updateWriterNameAndStdID(unsigned wrtID, const
                        std::string& name, const std::string&
                        stdId);

    void deleteCharRow(unsigned chID);
    void deleteWriter(unsigned wrtID);

    unsigned int getMaxWriterID();
    const char* getDBName() const;
    const char* getHostAddress() const;
    const char* getUserName() const;
    const char* getPassWord() const;
    bool getCharacterSet
        (std::vector<CharacterDB>& charSet);
    void getCharacterByWriter(unsigned wrtID,
        std::vector<CharacterDB>& charSet);
    void getWriterSet(std::vector<Writer>& wrtSet);
    void setNullWriterToCharacter(const
        std::vector<CharacterDB>& charSet);
};

```

bool isWriterExist(std::string name);

ตรวจสอบว่ามีชื่อผู้ใช้นี้ในระบบหรือไม่ในฐานข้อมูล

bool selectWriterByRow(std::string name,Writer& wrt);

เลือกดึงข้อมูลของผู้ใช้จากฐานข้อมูล

bool selectWriterByRow(std::string name);

เลือกดึงข้อมูลของผู้ใช้จากฐานข้อมูล

```
unsigned int selectWriterID(std::string name);
```

ให้รหัส Writer IDของผู้ใช้จากฐานข้อมูล

```
bool insertWriter(const Writer& wrt);
```

เพิ่มผู้ในระบบในฐานข้อมูล

```
unsigned int insertNewWriter(const Writer& wrt);  
void updateLstWrtTime(unsigned int wrtId,  
std::string lstTime);
```

เพิ่มผู้ในระบบในฐานข้อมูล

```
unsigned int insertCharacter(const CharacterDB& );
```

เพิ่มข้อมูลตัวอักษรในฐานข้อมูล

```
unsigned int getMaxWriterID();
```

ให้รหัส Writer ID สูงสุดของผู้ใช้จากฐานข้อมูล

```
void updateRecogLabel(unsigned int charId,  
std::string recogChar);
```

แก้ไขผลการ Recognition ของลายมือเขียนในฐานข้อมูล

```
void updateCharacter(const CharacterDB& chDB);
```

แก้ไขข้อมูลตัวอักษรในฐานข้อมูล

```
bool getCharacterSet(std::vector<CharacterDB>&);
```

ให้ข้อมูลตัวอักษรทั้งหมดในฐานข้อมูล

```
void deleteCharRow(unsigned chID);
```

ลบข้อมูลตัวอักษรในฐานข้อมูล

```
void updateWriter(const Writer& wrt);
```

แก้ไขผู้ในระบบในฐานข้อมูล

```
void deleteWriter(unsigned wrtID);
```

ลบผู้ในระบบในฐานข้อมูล

```
void getWriterSet(std::vector<Writer>& wrtSet);
```

ให้ผู้ในระบบทั้งหมดในฐานข้อมูล

```
void updateWriterNameAndStdID(unsigned wrtID, const  
std::string& name, const std::string& stdId);
```

แก้ไขข้อมูลชื่อ และรหัสนักศึกษาของผู้ใช้ ในฐานข้อมูล

4.2.8. ไม่มี Namespace

4.2.8.1. Class CurveListBuilder เก็บตัวอักษรลายมือเขียนเป็นลำดับของพิกัด (x, y) ที่เขียนบน Writing Pad ประกอบด้วยฟังก์ชันต่างๆ ดังนี้

`void setPointList(const MyPoint::PointList& pntLst)`

กำหนดค่าลำดับของพิกัด(x, y) ซึ่งเป็นอินพุทลายมือเขียน

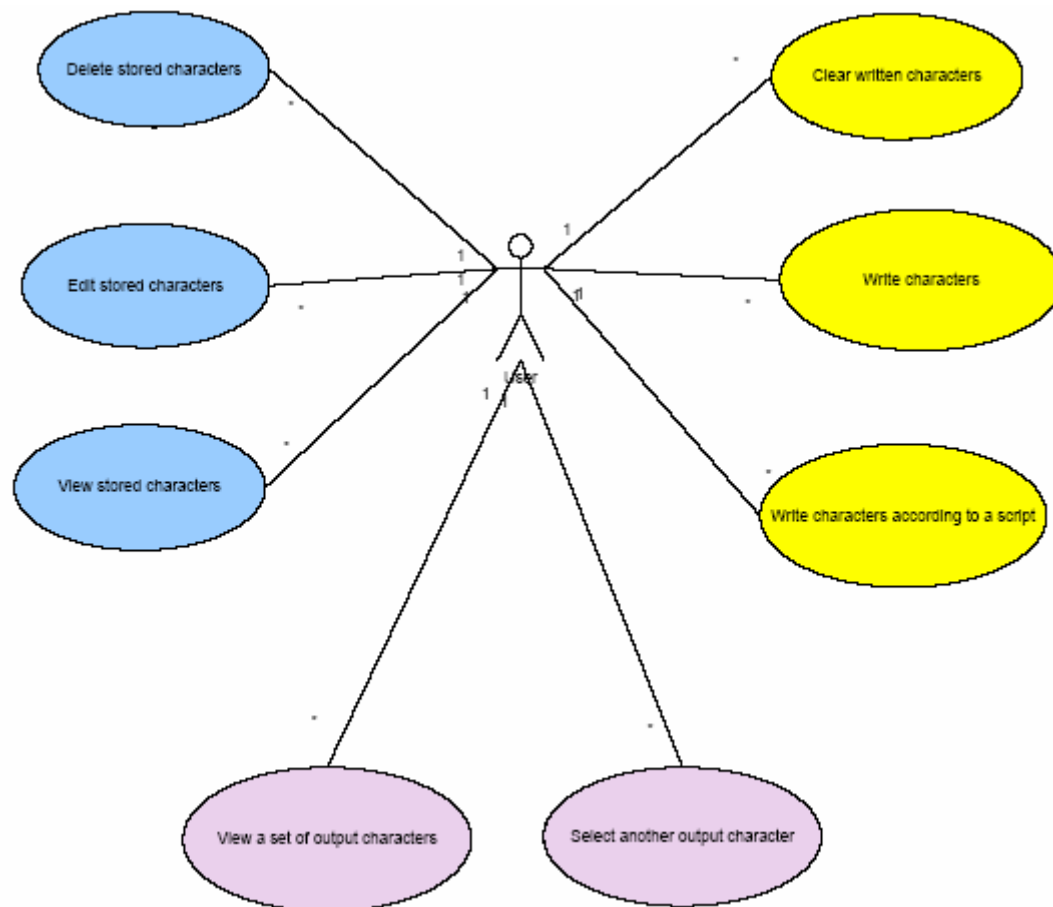
`const MyPoint::PointList& getPointList() const;`

ให้ค่าลำดับของพิกัด(x, y) ซึ่งเป็นอินพุทลายมือเขียน

4.2.8.2. Class ThaiRecognition คลาสติดต่อกับ โครงงานเดิม โดยเป็น dynamic link library ค่าต่างๆที่ได้ออกมาส่งผ่านทาง pointer

4.3 UML ของระบบ

4.3.1 การออกแบบ Use Case Diagram ของระบบ

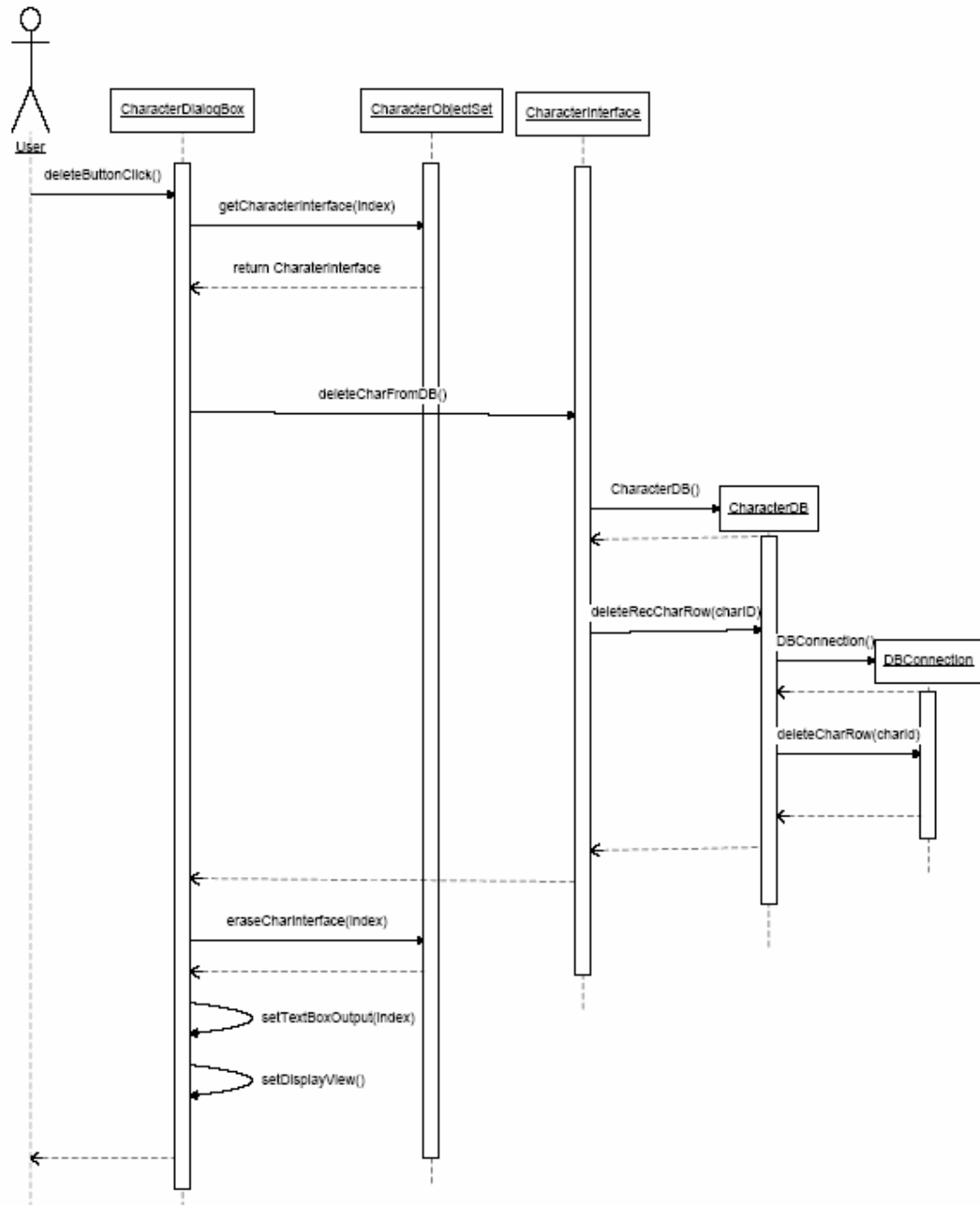


รูปที่ 4.2 แสดง Usecase Diagram ของระบบ

1. **Delete Stored Characters:** ลบข้อมูลตัวอักษรลายมือเขียนที่ละตัวอักษรในฐานข้อมูล
2. **Edit Stored Characters:** แก้ไขข้อมูลตัวอักษรลายมือเขียนที่ละตัวอักษรในฐานข้อมูล
3. **View Stored Characters:** ดูข้อมูลตัวอักษรลายมือเขียนที่ละตัวอักษรในฐานข้อมูล
4. **Clear Written Characters:** ตัวอักษรลายมือเขียนทั้งหมดใน Writing Pad
5. **Write Characters:** เขียนตัวอักษรลงใน Writing Pad
6. **Write Characters according to scripts:** เขียนตัวอักษรตามที่แสดงใน Script Window ลงใน Writing Pad
7. **View a set of output characters:** แสดงตัวอักษรทั้งหมดที่มีลักษณะใกล้เคียงกับตัวอักษรลายมือเขียน
8. **Select another output character:** เลือกตัวอักษรเอาท์พุทที่มีลักษณะใกล้เคียงกับตัวอักษรลายมือเขียน

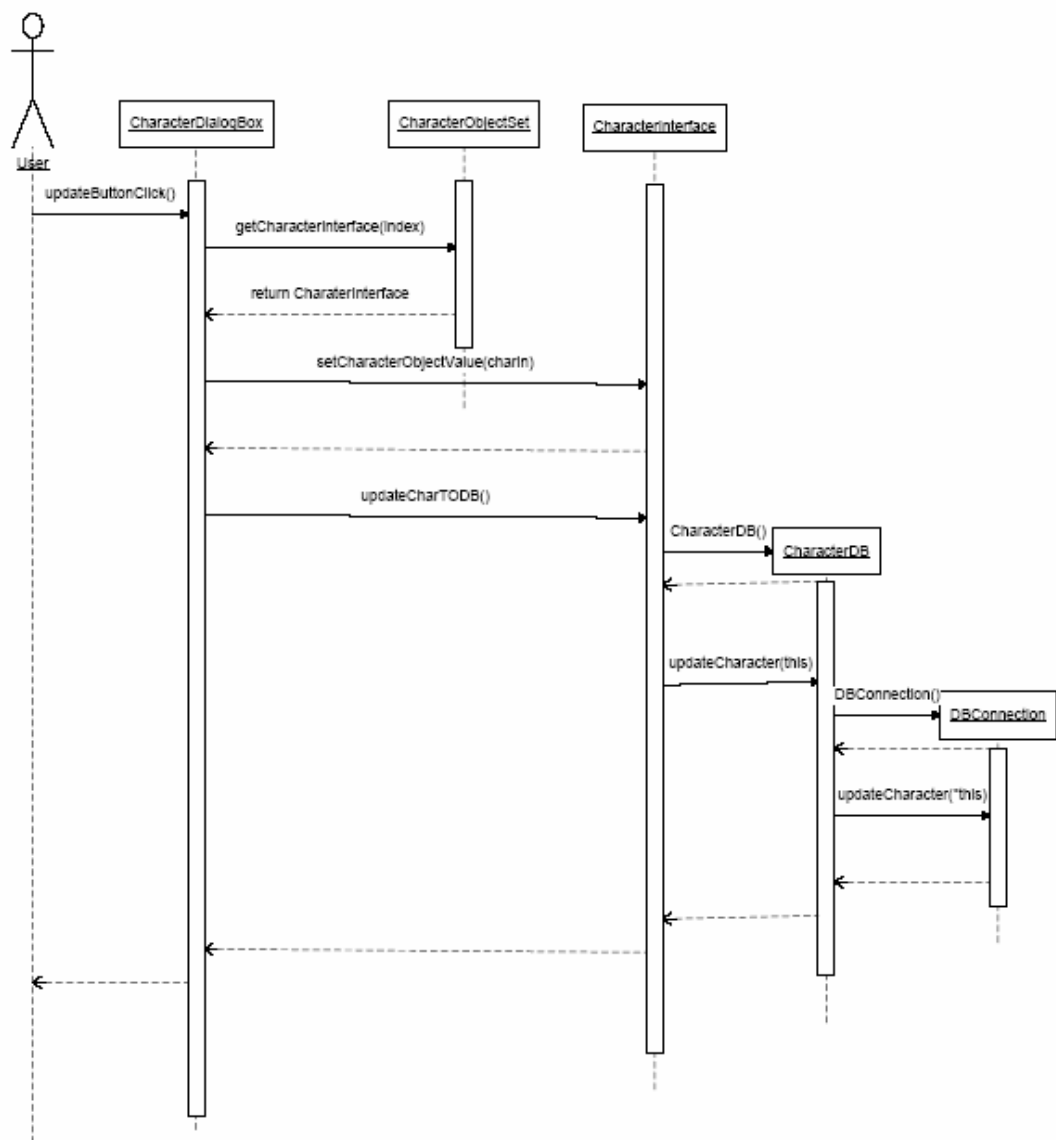
4.3.2 การออกแบบ Sequence Diagram ของระบบ

4.3.2.1. Delete Stored Characters



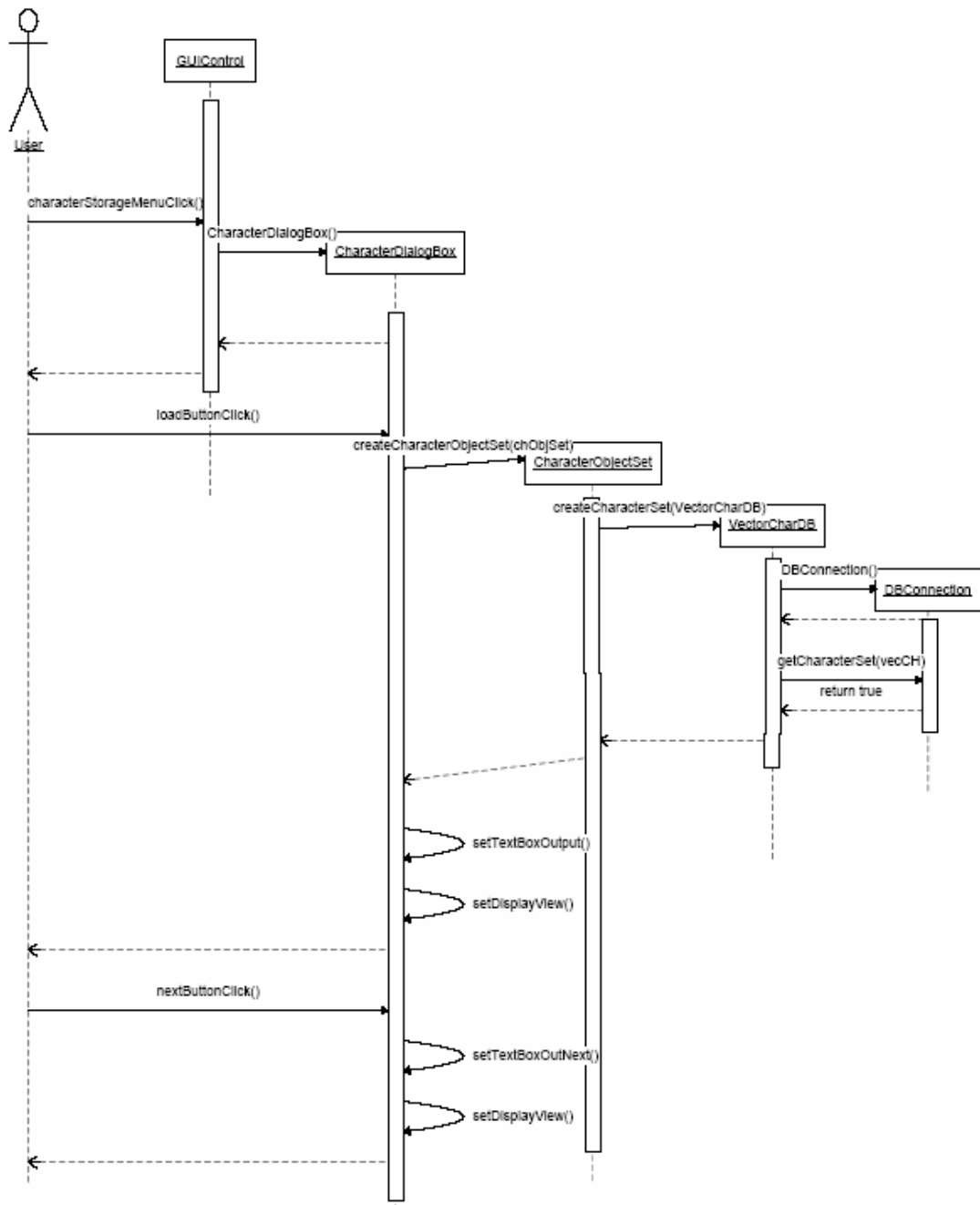
รูปที่ 4.3 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้ลบตัวอักษรออกจากฐานข้อมูล

4.3.2.2. Edit Stored Characters



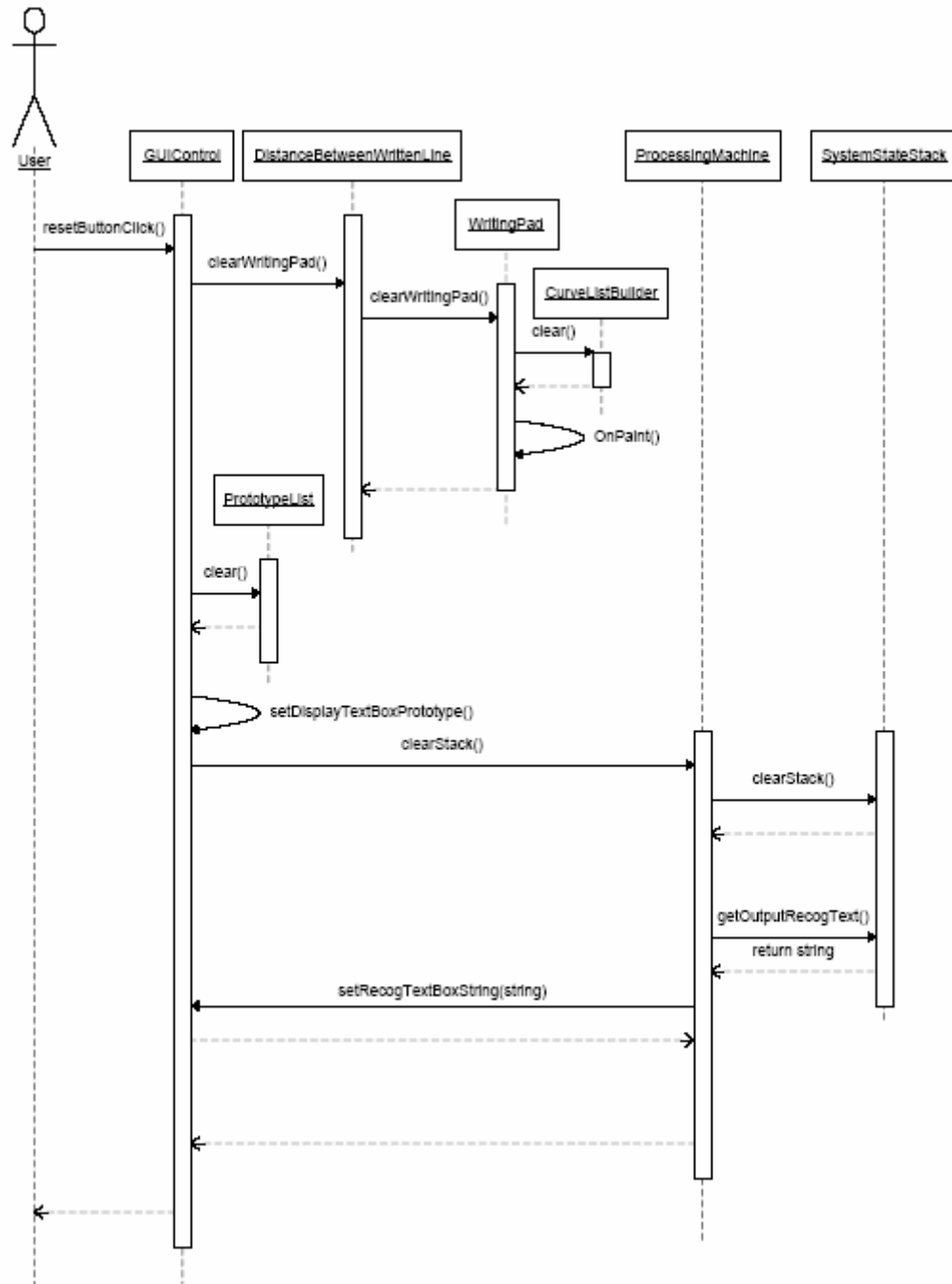
รูปที่ 4.4 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้แก้ไขข้อมูลตัวอักษรในฐานข้อมูล

4.3.2.3. View Stored Characters



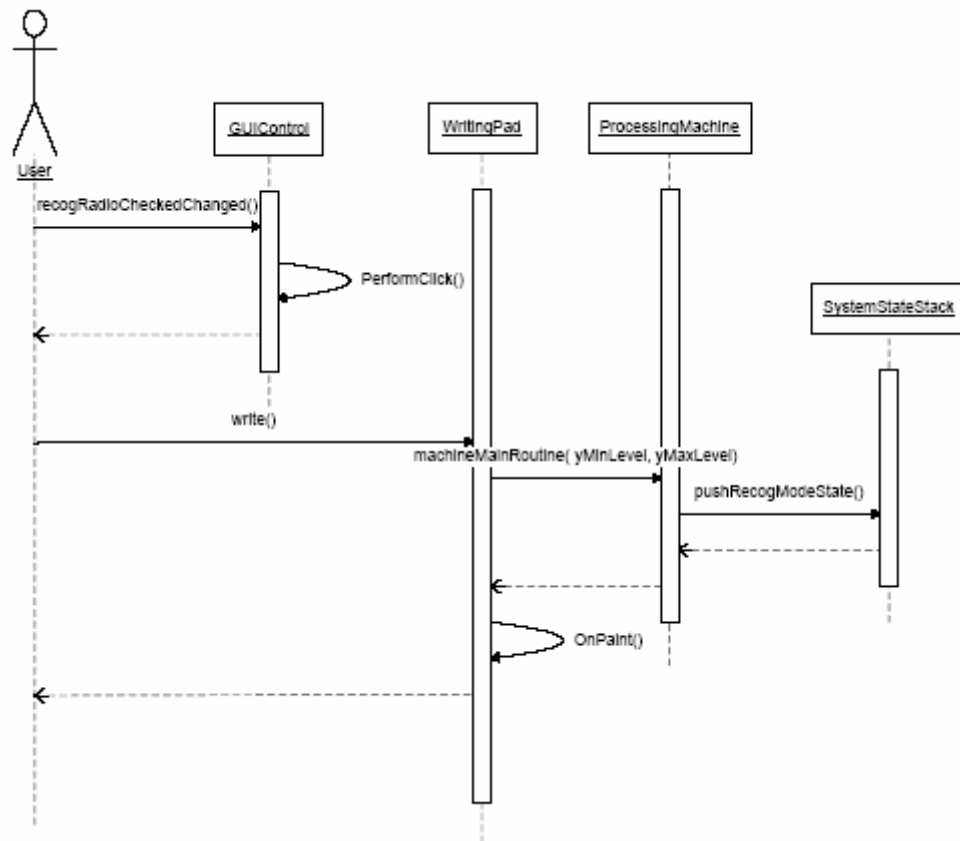
รูปที่ 4.5 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้เรียกดูข้อมูลตัวอักษรในฐานข้อมูล

4.3.2.4. Clear Written Characters



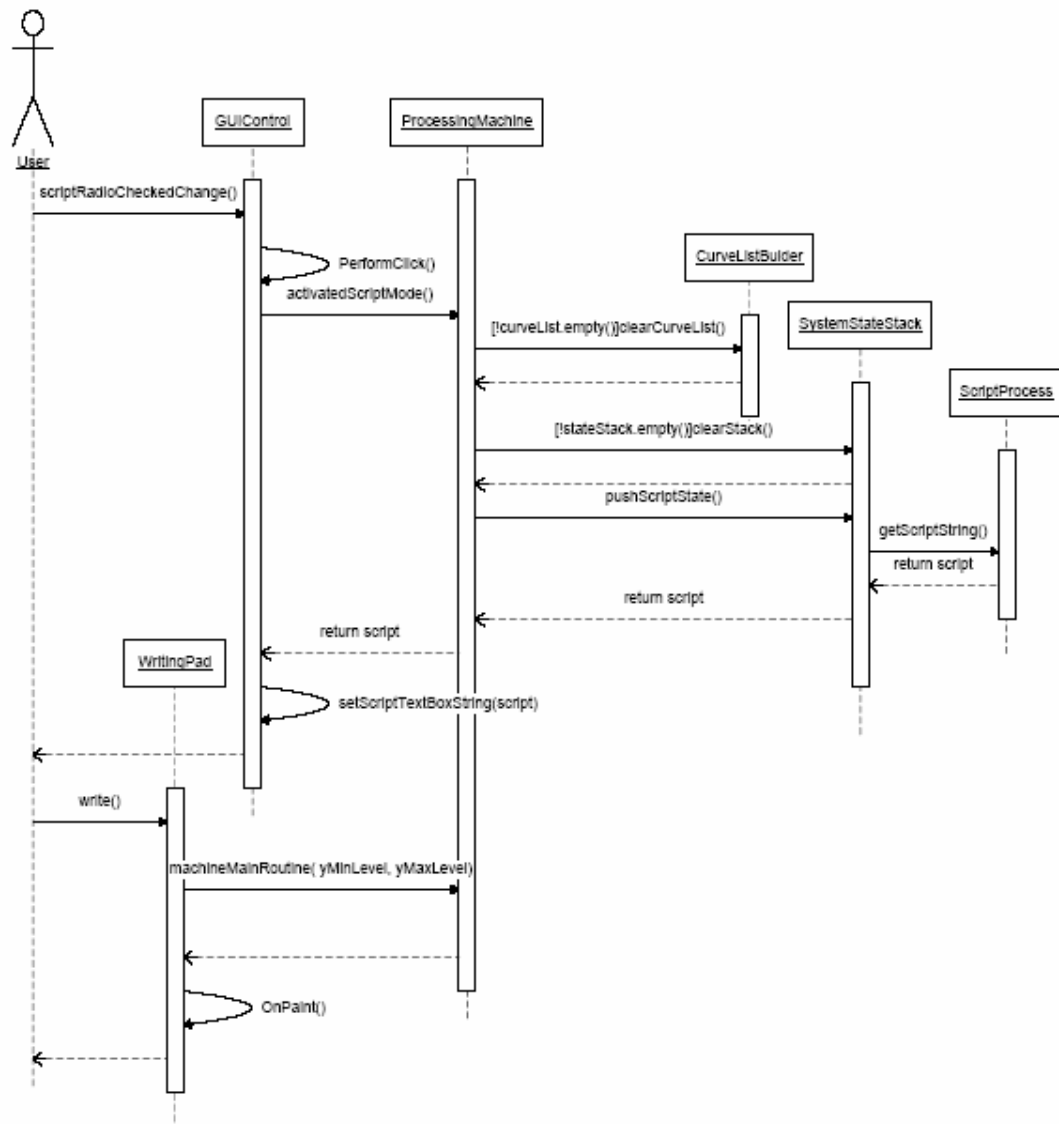
รูปที่ 4.6 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้งานต้องการรีเซตระบบใหม่

4.3.2.5. Write Characters



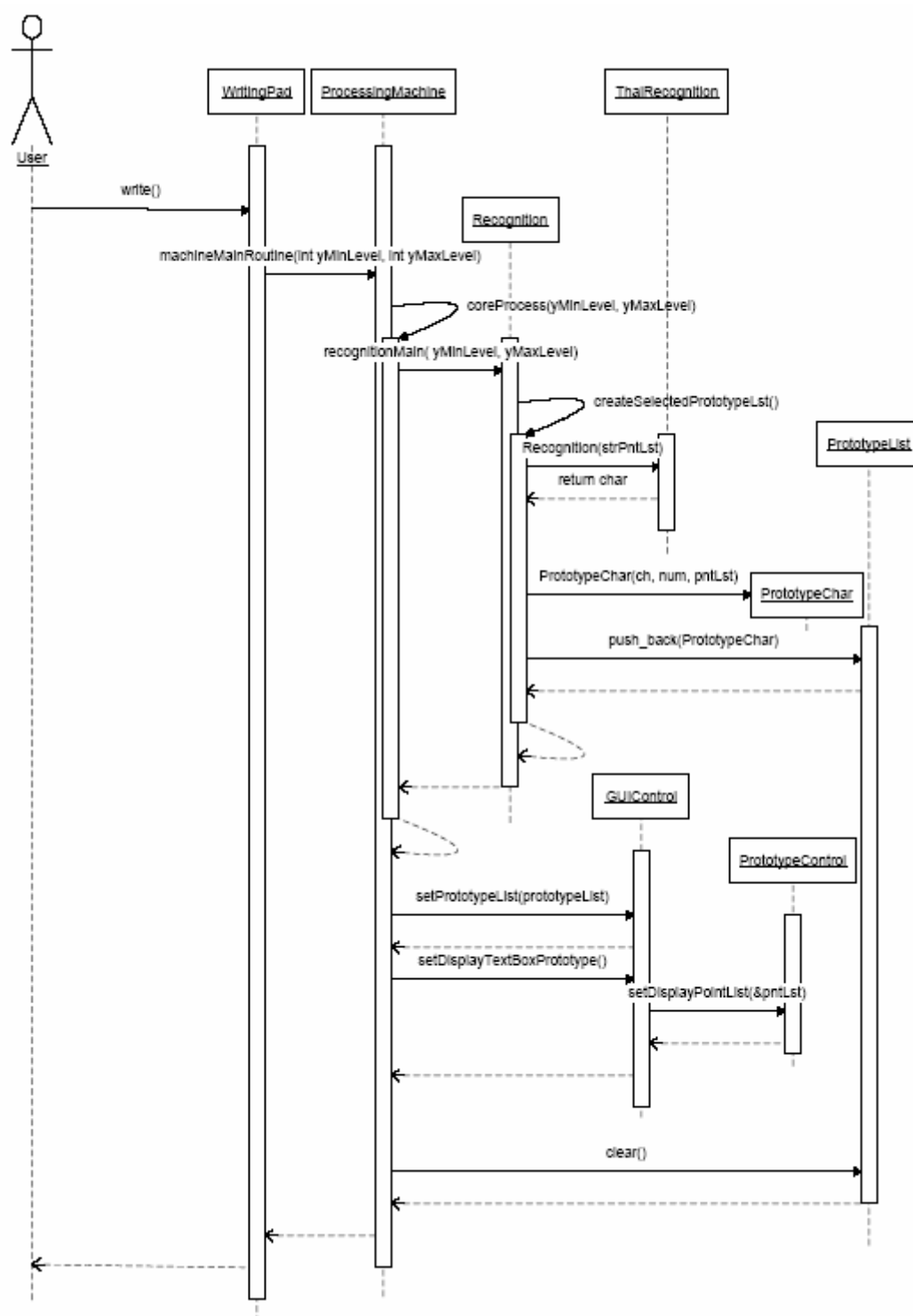
รูปที่ 4.7 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้งานต้องการเขียนตัวอักษรใดๆ

4.3.2.6. Write Characters according to scripts:



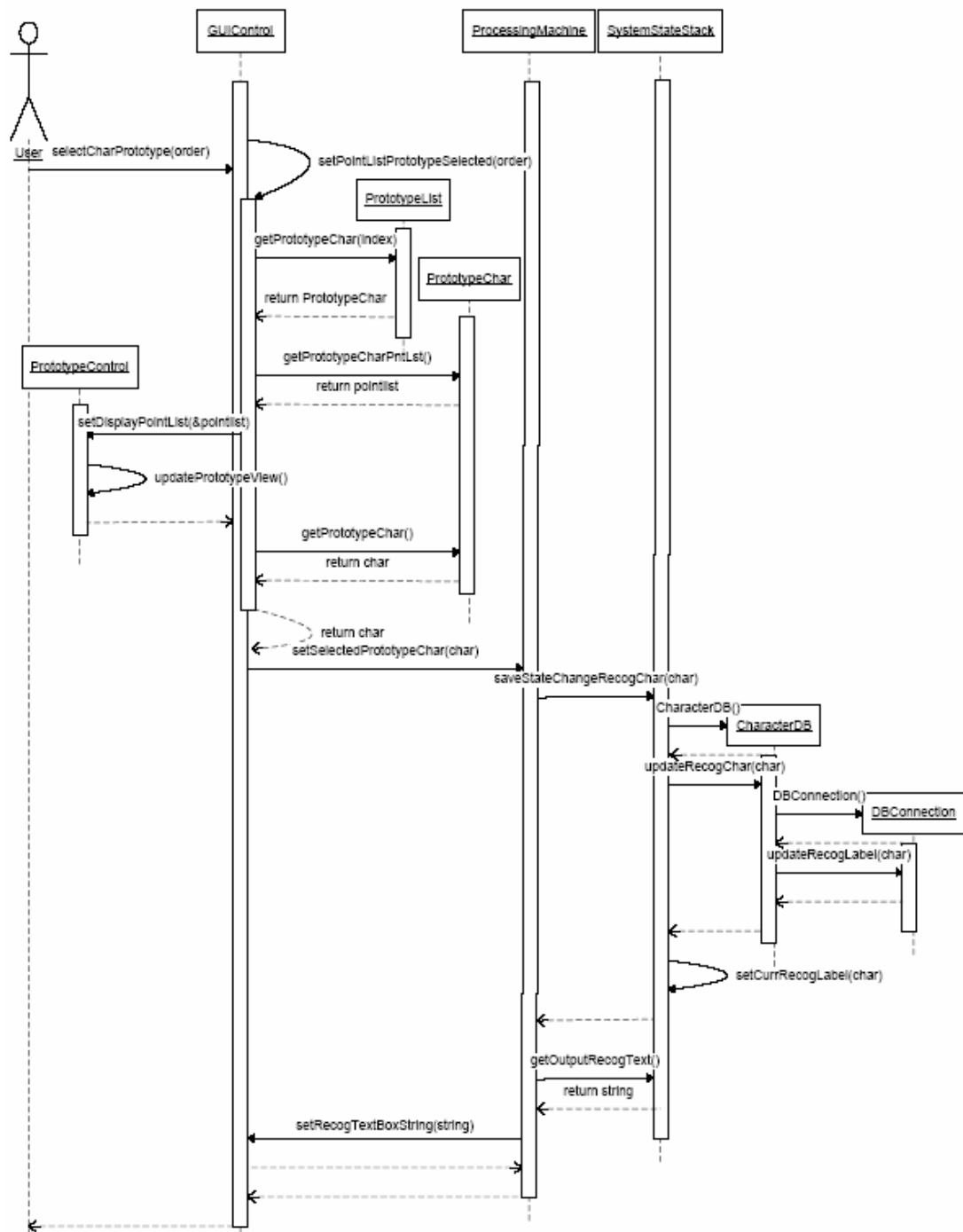
รูปที่ 4.8 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้งานต้องการเขียนตัวอักษรตามสคริปต์

4.3.2.7. View a set of output characters



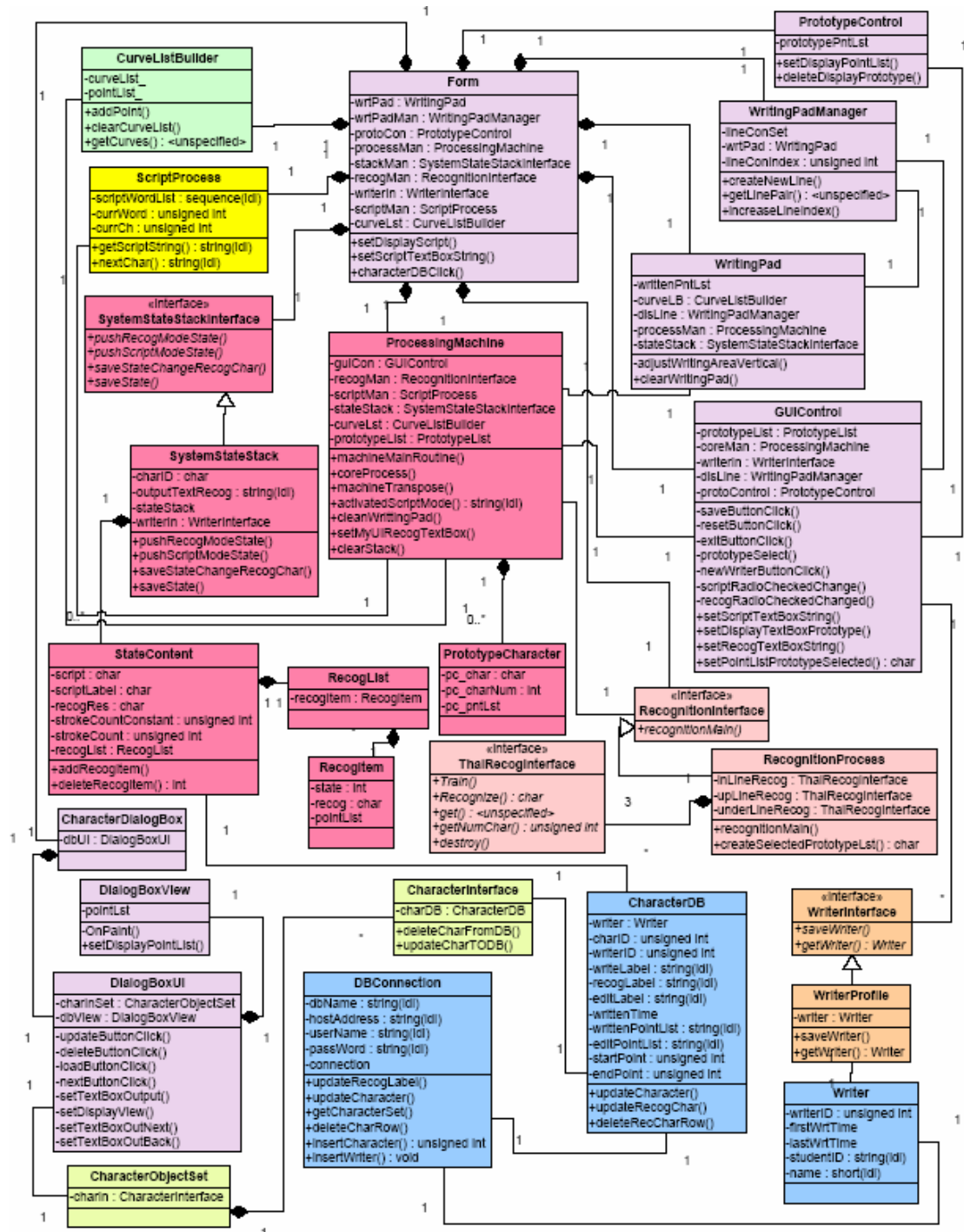
รูปที่ 4.9 แสดง Sequence Diagram ของระบบซึ่งแสดงผลตัวอักษรตัวพิมพ์ที่ใกล้เคียงกับลายมือเขียนมากที่สุด 5 อันดับแรก

4.3.2.8. Select another output character



รูปที่ 4.10 แสดง Sequence Diagram ของระบบเมื่อผู้ใช้เลือกตัวอักษรตัวพิมพ์ตัวอื่นๆ

4.3.3 การออกแบบ Class Diagram ของระบบ



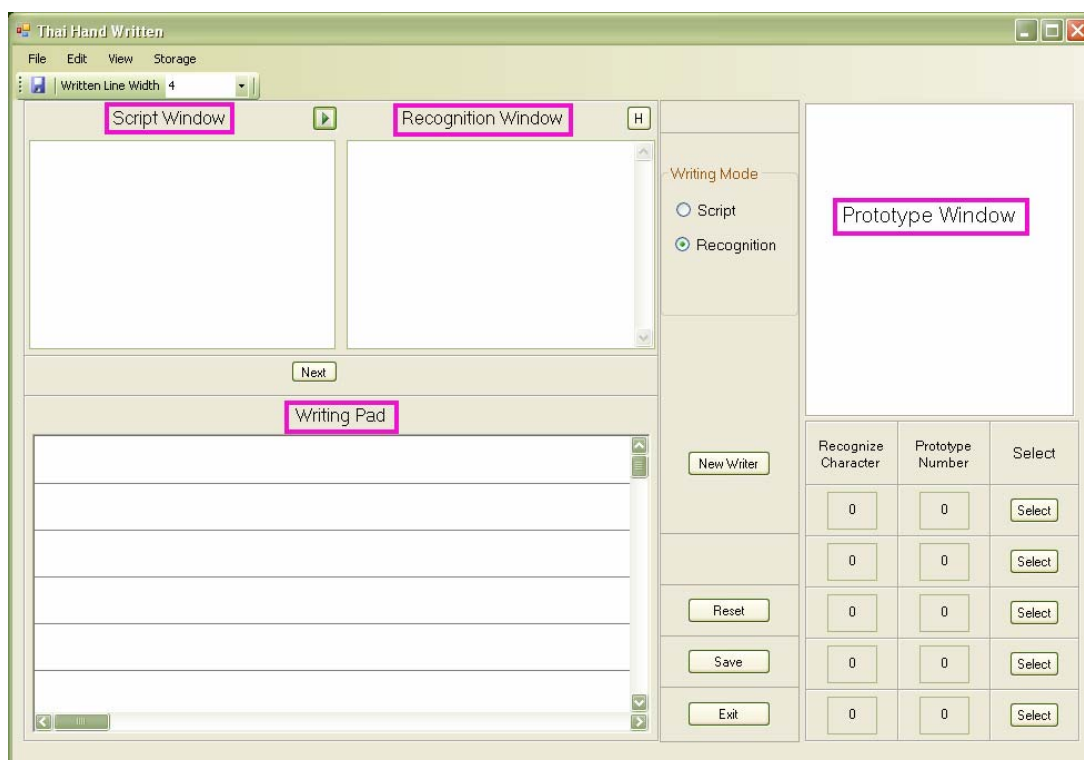
รูปที่ 4.11 แสดง Class Diagram ของระบบ

4.4 การออกแบบหน้าจอส่วนติดต่อกับผู้ใช้งานระบบ

เนื่องจากระบบอ่านลายมือเขียนประกอบด้วยการติดต่อกับผู้ใช้งานเป็น 3 รูปแบบหลัก ดังนี้

1. การเขียนตัวอักษรตามที่ระบบแสดง(เขียนตาม script)
2. การเขียนตัวอักษรใดๆ
3. การดูข้อมูลในฐานข้อมูล

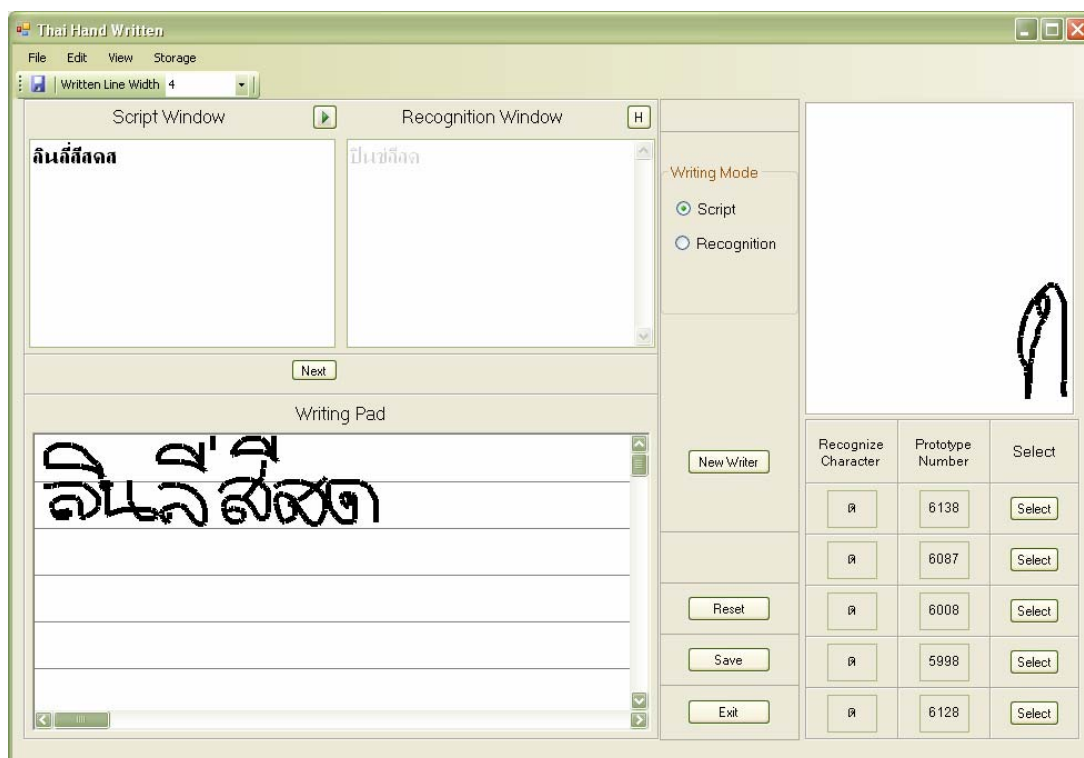
หน้าจอติดต่อกับผู้ใช้งานระบบจึงถูกออกแบบให้มีลักษณะดังนี้



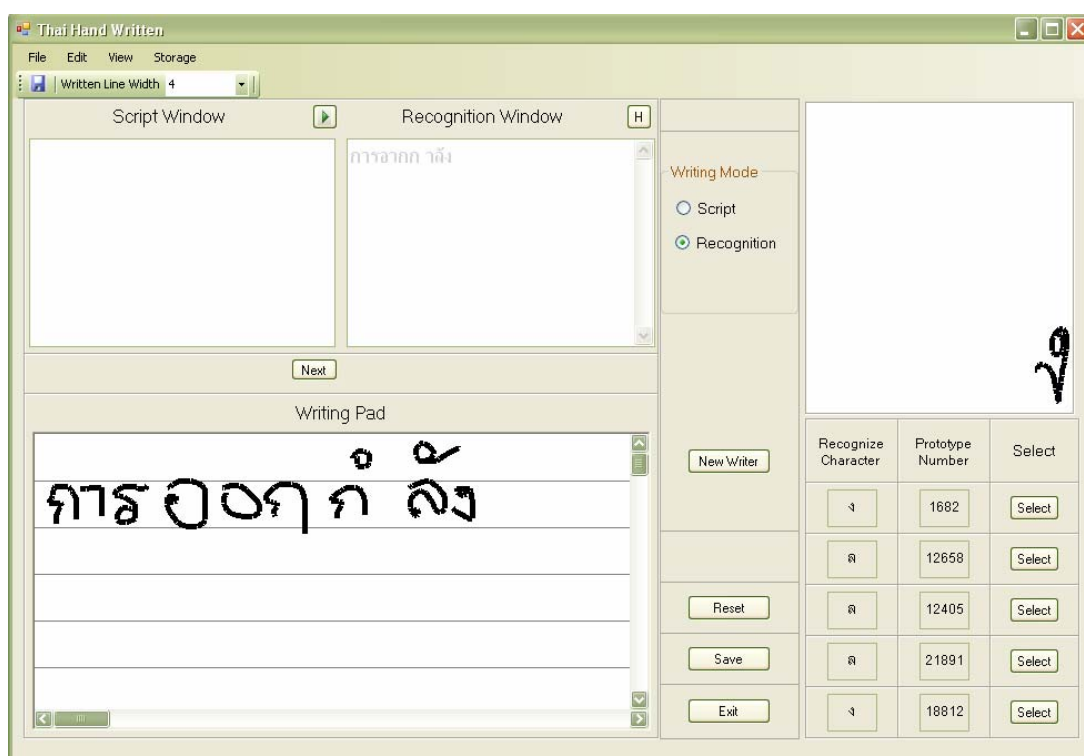
รูปที่ 4.12 แสดง User Interface หลักของระบบ

ในหน้าต่างประกอบด้วยหน้าต่างย่อยซึ่งทำหน้าที่ต่าง ๆ กันดังนี้

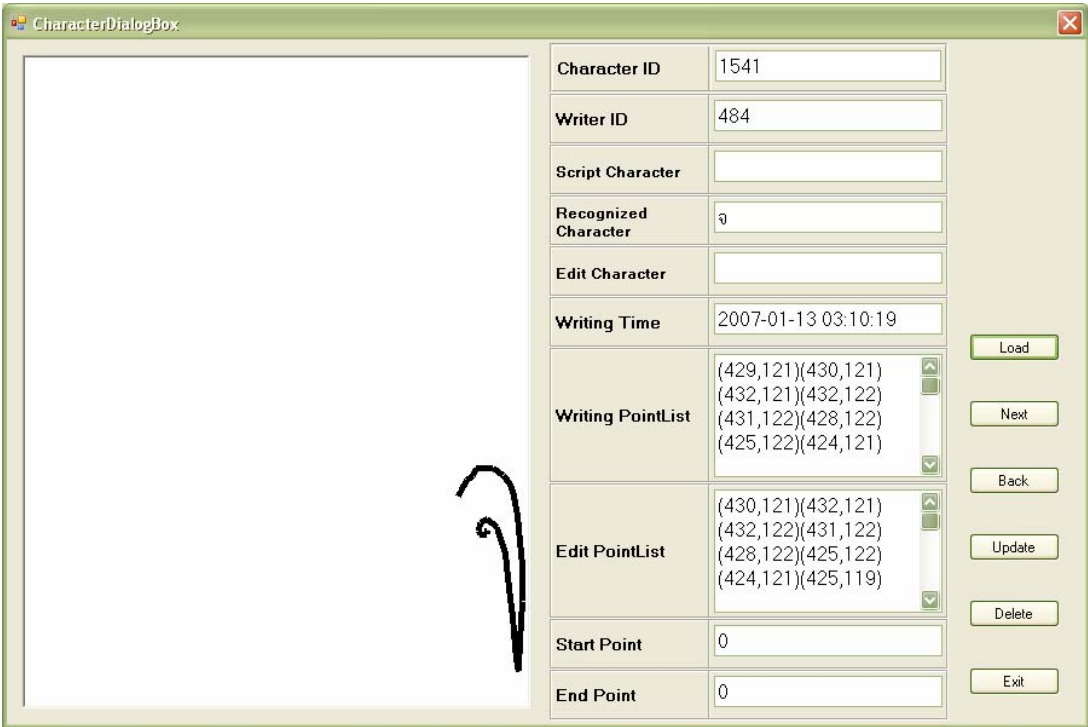
1. Script Window แสดงตัวอักษรที่ต้องการให้ผู้ใช้งานระบบเขียนตาม
2. Recognition Window แสดงตัวอักษรตัวพิมพ์ซึ่งเป็นเอาพุทของระบบ
3. Prototype Window แสดงตัวอักษรลายมือเขียนที่ใช้เป็นตัวอักษรตัวอย่างให้ระบบ



รูปที่ 4.13 แสดง User Interface หลักของระบบเมื่อผู้ใช้เขียนตัวอักษรตามสคริปต์



รูปที่ 4.14 แสดง User Interface หลักของระบบเมื่อผู้ใช้เขียนตัวอักษรใดๆ



The screenshot shows a window titled "CharacterDialogBox" with a large empty canvas on the left and a form on the right. A handwritten Thai character "จ" is on the canvas. The form contains the following fields:

Character ID	1541
Writer ID	484
Script Character	
Recognized Character	จ
Edit Character	
Writing Time	2007-01-13 03:10:19
Writing PointList	(429,121)(430,121) (432,121)(432,122) (431,122)(428,122) (425,122)(424,121)
Edit PointList	(430,121)(432,121) (432,122)(431,122) (428,122)(425,122) (424,121)(425,119)
Start Point	0
End Point	0

Buttons on the right: Load, Next, Back, Update, Delete, Exit.

รูปที่ 4.15 แสดง User Interface ของระบบเมื่อผู้ใช้เรียกดูข้อมูลตัวอักษรจากฐานข้อมูล

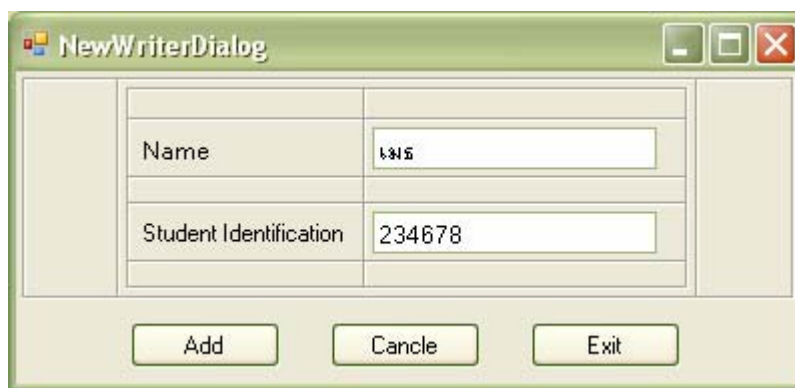


The screenshot shows the same "CharacterDialogBox" window with a different form layout. The fields are:

Writer Identification	597
Student Identification	
Name	เกด
First Writing Time	2007-02-02 23:50:47
Last Writing Time	2007-02-02 23:50:47

Buttons on the right: Load, Update, Delete, Next, Back, Exit.

รูปที่ 4.16 แสดง User Interface ของระบบเมื่อผู้ใช้เรียกดูข้อมูลผู้ใช้งานระบบจากฐานข้อมูล



รูปที่ 4.17 แสดง User Interface ของระบบเมื่อเพิ่มผู้ใช้งานระบบเข้าสู่ฐานข้อมูล

4.5 การพัฒนาระบบอ่านลายมือเขียนภาษาไทย

เนื่องจากการพัฒนาใช้เครื่องมือ .Net Framework โดยใช้ภาษา CLI ซึ่งทำให้สามารถเรียกใช้ library ได้ทั้งของ C++ และ C# ทำให้ส่วน user interface พัฒนาได้อย่างรวดเร็ว แต่ C++ และ C# จัดการกับหน่วยความจำต่างกัน ข้อมูลของ C++ จะถูกเก็บไว้ในหน่วยความจำที่เรียกว่า unmanaged memory ซึ่งก็คือระบบจะไม่จัดการคืน memory ให้นอกจากผู้ใช้งานจะต้องรับผิดชอบการคืน memory เอง ต่างจากภาษา C# ซึ่งข้อมูลของ C# จะถูกเก็บไว้ในหน่วยความจำที่เรียกว่า managed memory ที่ผู้ใช้งานไม่รับผิดชอบการคืนหน่วยความจำ เพราะระบบจะจัดการคืนหน่วยความจำให้เมื่อข้อมูลดังกล่าวถูกเลิกใช้

การจัดการกับหน่วยความจำต่างกัดังกล่าวของ C++ และ C# ทำให้เกิดปัญหาในการอ้างอิงถึงข้อมูลระหว่างกัน เพราะข้อมูลถูกเก็บไว้ในหน่วยความจำที่ต่างกันและมองไม่เห็นกัน ทำให้ต้องศึกษาเพิ่มเติมในส่วนนี้ค่อนข้างมาก

นอกจากนี้ ภาษาทั้งสอง C++ และ C# ก็ใช้รหัสตัวอักษรที่ต่างกันโดย C++ ใช้อักษร ASCII ในขณะที่ C# ใช้อักษร UNICODE จึงต้องมีการแปลงรหัสอักษรไปมาระหว่างการประมวลผลโปรแกรม นั่นคือ ส่วนที่ติดต่อกับผู้ใช้งานระบบใช้รหัส UNICODE ขณะที่ส่วน business process และ storage ใช้อักษร ASCII

4.5.1 การพัฒนาโปรแกรมเพื่อจัดการกับลายมือเขียนที่บรรทัดต่างๆ

จัดการกับลายมือเขียนที่บรรทัดต่างๆ โดยสร้างไฟล์เก็บตัวอักษรตัวอย่างที่ตำแหน่งกัน (โปรโตไทป์ของตัวอักษรที่ตำแหน่งกัน) ไว้ต่างไฟล์กัน และสร้าง instant ของ class ThaiRecognizer สาม instant โหลดข้อมูลตัวอักษรตัวอย่างจากไฟล์ที่ต่างกัดังกล่าว เพื่อจัดการกับลายมือเขียนที่ตำแหน่งต่างๆ

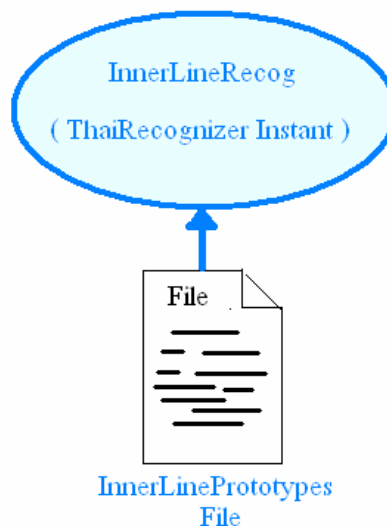
```

character.txt - Notepad
File Edit Format View Help
n-0
$xy
(157,133)(162,123)(166,113)(169,103)(172,94)(172,90)(173,87)(173,85)(173,84)(170,83)(167,82)
(162,82)(159,80)(157,79)(156,79)(156,78)(156,76)(156,74)(156,72)(160,71)(164,69)(168,68)(173,67)
(179,67)(186,67)(192,67)(198,67)(202,67)(205,69)(209,71)(211,74)(212,78)(214,83)(215,89)(215,95)
(216,103)(216,111)(216,120)(216,130)(216,137)(216,145)(214,151)(214,155)(212,158)(212,162)
(211,162)
#
n-1
$xy
(146,136)(147,130)(148,123)(149,117)(151,111)(151,107)(151,103)(151,101)(151,100)(150,100)
(148,100)(145,100)(142,100)(138,100)(133,100)(129,100)(125,100)(121,100)(119,100)(118,100)
(118,99)(118,97)(120,94)(123,91)(126,87)(131,84)(136,81)(140,79)(142,77)(143,77)(144,77)(145,77)
(147,77)(148,77)(150,77)(153,77)(157,77)(162,77)(166,77)(170,79)(174,81)(177,83)(179,86)(181,89)
(182,94)(182,100)(182,104)(182,109)(182,115)(182,120)(182,125)(182,129)(182,134)(182,138)
(182,141)(182,144)(182,146)(182,147)
#
n-2
$xy
(145,161)(145,160)(145,159)(145,158)(145,157)(145,156)(145,155)(145,154)(145,152)(145,150)
(146,146)(146,142)(146,136)(146,127)(146,117)(146,108)(146,98)(146,90)(146,85)(146,83)(145,83)
(142,83)(138,83)(131,83)(125,83)(119,84)(114,84)(111,85)(111,84)(112,81)(116,78)(122,72)(131,67)
(139,61)(148,55)(156,51)(165,48)(173,44)(179,43)(185,43)(191,43)(194,43)(197,43)(199,45)(200,48)

```

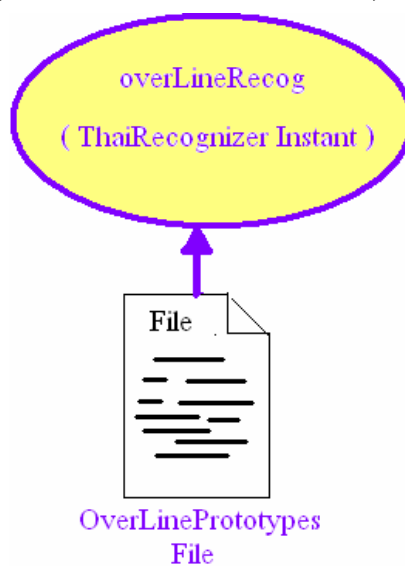
รูปที่ 4.18 แสดงไฟล์สำหรับเก็บตัวอักษรตัวอย่าง (ตัวอักษรprototype)

1. ตัวอักษรที่อยู่ในบรรทัด สร้าง innerLineRecog instant : โหลดตัวอักษรจากไฟล์
ตัวอักษรตัวอย่างในบรรทัด (โปรโตไทป์ของอักษรในบรรทัด)



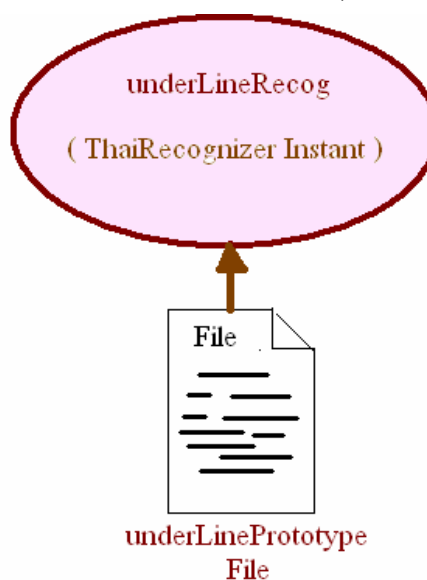
รูปที่ 4.19 แสดงการโหลดตัวอักษรตัวอย่าง (ตัวอักษรprototype) จากไฟล์สำหรับเก็บ
ตัวอักษรตัวอย่างในบรรทัด

2. ตัวอักษรบนบรรทัด สร้าง overLineRecog instant : : โหลดตัวอักษรจากไฟล์
ตัวอักษรตัวอย่างบนบรรทัด (โปรโตไทป์ของอักษรบนบรรทัด)

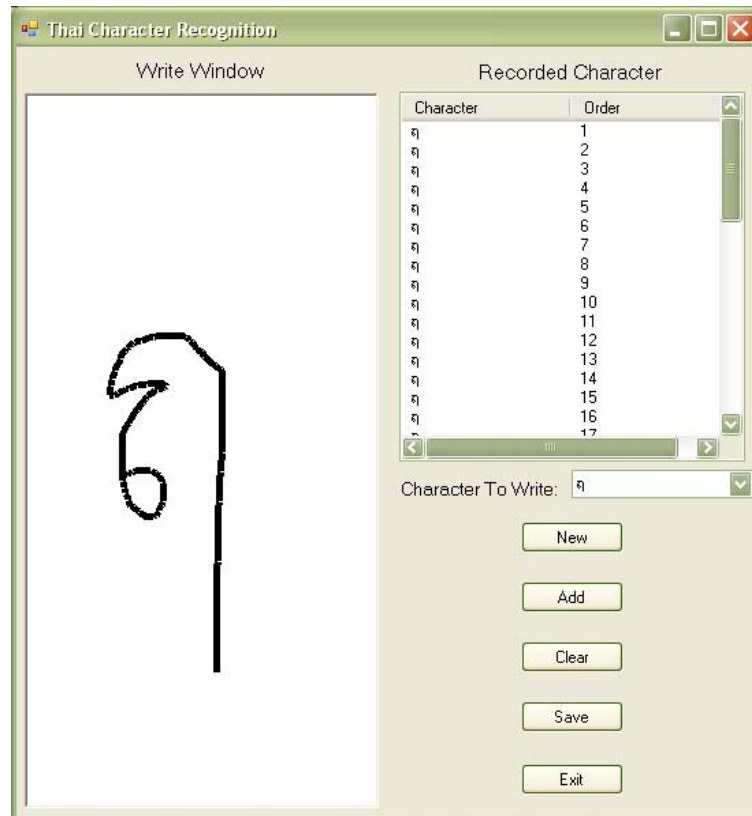


รูปที่ 4.20 แสดงการโหลดตัวอักษรตัวอย่าง (ตัวอักษรprototype) จากไฟล์สำหรับเก็บ
ตัวอักษรตัวอย่างเหนือเส้นบรรทัด

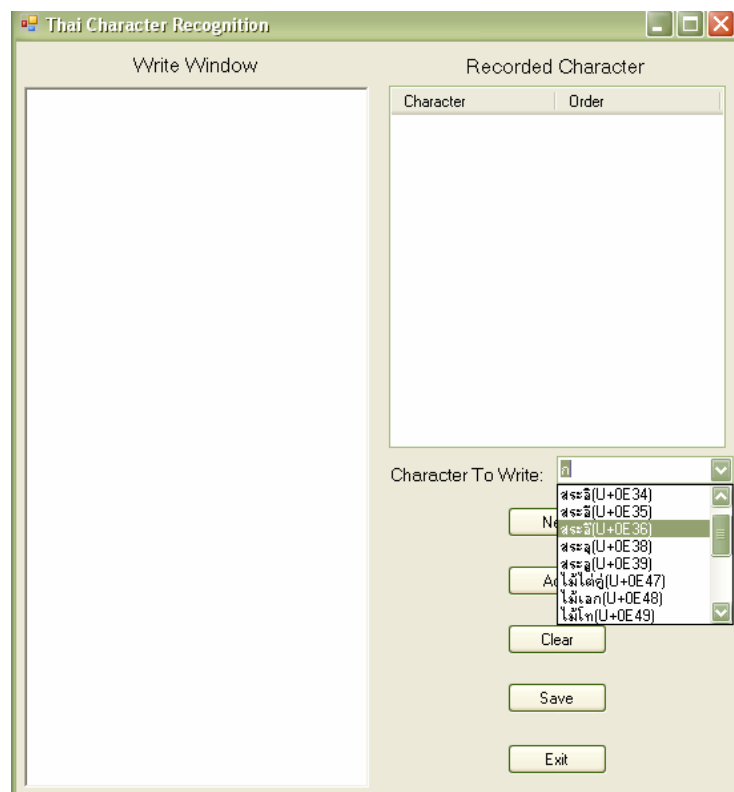
3. ตัวอักษรใต้บรรทัด สร้าง underLineRecog instant : : โหลดตัวอักษรจากไฟล์
ตัวอักษรตัวอย่างใต้บรรทัด (โปรโตไทป์ของอักษรใต้บรรทัด)



รูปที่ 4.21 แสดงการโหลดตัวอักษรตัวอย่าง (ตัวอักษรprototype) จากไฟล์สำหรับเก็บ
ตัวอักษรตัวอย่างใต้เส้นบรรทัด



รูปที่ 4.23 แสดงโปรแกรมรวบรวมตัวอักษรลายมือเขียนตัวอย่างในบรรทัดเพิ่มเติม



รูปที่ 4.24 แสดงโปรแกรมรวบรวมตัวอักษรลายมือเขียนตัวอย่างบนและได้บรรทัดเพิ่มเติม

4.5.3 พัฒนารฐานข้อมูล สำหรับเก็บรวบรวมข้อมูลลายมือเขียน โดยแบ่งออกเป็น 2 ตารางซึ่งประกอบด้วยคอลัมน์ต่างๆดังนี้

4.5.3.1 ตาราง CharacterDB

CHARACTER_ID: รหัสประจำตัวอักษร

WRITER_ID: รหัสประจำตัวผู้เขียนตัวอักษร

WRITE_LABEL: ตัวอักษรที่ผู้ใช้งานโปรแกรมต้องการจะเขียนหรือตัวอักษรที่กำหนดใน Script Window

RECOG_LABEL: ตัวอักษรตัวพิมพ์เอापุทที่ได้จากการรู้จำตัวอักษร

EDIT_LABEL: ตัวอักษรตัวพิมพ์เอापุทจริงควรจะเป็น

WRITTEN_TIME: เวลาที่เขียนตัวอักษร

WRITTEN_POINTLIST: เซตของพิกัด (x, y) ในการเขียนตัวอักษร

EDIT_POINTLIST: เซตของพิกัด (x, y) ในการเขียนตัวอักษรที่ได้รับการแก้ไขแล้ว

START_POINT: จุดเริ่มต้นการเขียนเป็น index ซึ่งไปยังพิกัดต่างๆในเซต

END_POINT: จุดสิ้นสุดการเขียนเป็น index ซึ่งไปยังพิกัดต่างๆในเซต

4.5.3.2 ตาราง Writer

WRITER_ID: รหัสประจำตัวผู้เขียนตัวอักษร

FIRST_WRITTEN_TIME: เวลาที่ผู้เขียนเริ่มเขียนตัวอักษรตัวแรก

LAST_WRITTEN_TIME: เวลาที่ผู้เขียนเขียนตัวอักษรตัวสุดท้าย

STUDENT_ID: รหัสนักศึกษา

NAME: ชื่อ

←T→	WRITER_ID	FIRST_WRITTEN_TIME	LAST_WRITTEN_TIME	NAME
☐  	1	2006-09-17 21:21:25	2006-09-17 21:24:58	anonymous1
☐  	2	2006-09-17 21:25:14	2006-09-17 21:30:37	anonymous2
☐  	3	2006-09-17 21:31:32	2006-09-17 21:32:11	anonymous3
☐  	4	2006-09-17 21:32:32	2006-09-17 21:33:15	anonymous4
☐  	5	2006-09-17 21:33:24	2006-09-17 21:34:11	anonymous5
☐  	6	2006-09-17 21:34:16	2006-09-17 21:34:51	anonymous6

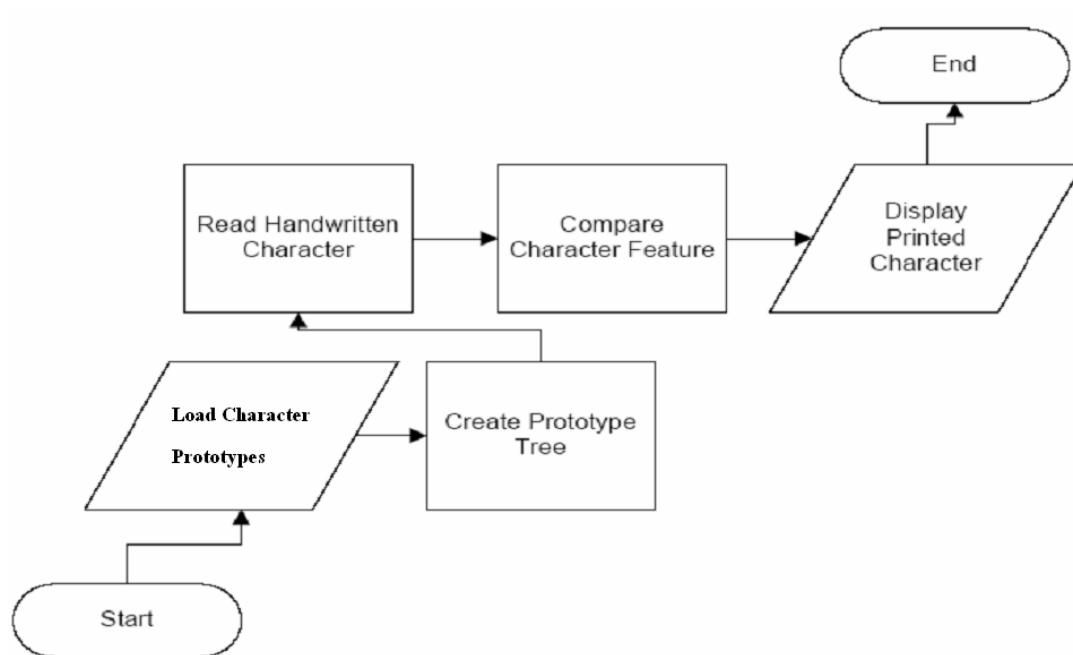
รูปที่ 4.25 แสดงตารางเก็บข้อมูลเกี่ยวผู้ใช้งานระบบ

←T→	CHARACTER_ID	WRITER_ID	WRITTEN_TIME	WRITTEN_POINTLIST
☐  	1	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	2	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	3	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	4	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	5	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	6	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	7	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	8	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	9	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	10	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	11	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	12	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	13	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	14	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	15	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	16	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	17	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	18	1	2006-09-17 21:24:41	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	19	1	2006-09-17 21:24:42	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	20	1	2006-09-17 21:24:42	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	21	1	2006-09-17 21:24:42	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	22	1	2006-09-17 21:24:42	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...
☐  	23	1	2006-09-17 21:24:42	(93,264)(93,261)(94,257)(96,254)(98,250)(101,248)(...

รูปที่ 4.26 แสดงตารางเก็บข้อมูลลายมือเขียน

4.5.4 ขั้นตอนการประมวลผลของโปรแกรม

1. โหลดตัวอักษรตัวอย่างขึ้นมาจากไฟล์ (โปรโตไทป์ของตัวอักษร)
2. สร้าง tree ซึ่งเป็น data structure สำหรับเก็บข้อมูลคุณลักษณะเด่นของตัวอักษรแต่ละตัวที่โหลดมาจากไฟล์
3. นำลักษณะเด่นของตัวอักษรอินพุต เปรียบเทียบกับลักษณะเด่นของตัวอักษรตัวอย่าง (โปรโตไทป์ของตัวอักษร) ทั้งหมด เพื่อหาตัวอักษรที่มีลักษณะเด่นใกล้เคียงกัน
(ทฤษฎีและกลไกการทำงานของระบบส่วนนี้ อาจารย์คำเฟื้อค บุญนะดี เป็นผู้ศึกษาและพัฒนา)



รูปที่ 4.27 แสดงขั้นตอนการประมวลผลของโปรแกรม